

optbench

A walkthrough of the thesis software artifact: how every file works

A study guide for the student. Project: fair benchmarking and visualization of optimizers for language-model fine-tuning.

Subject: optbench — a testbed for a fair comparison of optimizers (SGD+momentum, AdamW, Lion, Sophia) when fine-tuning language models with LoRA, and for the correct visualization of the loss landscape.

The document describes every file of the project: its purpose, its key lines, its links to other modules, and its methodological meaning.

Contents

- [How to read this document](#)
- [The big picture: what scientific problem the program solves](#)
- [The problem](#)
- [What this program does](#)
- [The fairness invariant](#)
- [Repository map](#)
- [The separation principle: library vs. scripts](#)
- [Control flow: what calls what](#)
- [The key data object: RunResult](#)
- [Layer 1. Project packaging](#)
- [Layer 2. Configuration](#)
- [Layer 3. Experiment descriptions \(configs/experiments/\)](#)
- [Layer 4. Data](#)
- [Layer 5. Models](#)
- [Layer 6. Optimizers](#)
- [Layer 7. Training](#)
- [Layer 8. Tuning \(RQ1/RQ2\)](#)
- [Layer 9. Experiment orchestration](#)
- [Layer 10. Analysis](#)
- [Layer 11. Loss landscape \(RQ3\)](#)
- [Layer 12. Infrastructure](#)
- [Layer 13. Scripts \(entry points\)](#)
- [Layer 14. Tests](#)

- [Results: how to read them](#)
 - [Three end-to-end scenarios](#)
 - [Strengths and weaknesses: what to say at the defence](#)
 - [Glossary](#)
 - [Self-check exercises](#)
-

How to read this document

The document is built "bottom-up": first the big picture and the scientific problem, then a walkthrough of every file in the order in which one layer builds on another, and at the end the end-to-end execution scenarios, where everything comes together.

Every file is presented with the same structure:

- **Path and role** — where it lives and what it is responsible for;
- **What's inside** — a line-by-line walkthrough of the key places;
- **Links** — who calls it and whom it calls;
- **What to pay attention to** — subtleties, mistakes, and questions you may be asked at the defence.

If you are reading this for the first time, work through the "Big picture" and "Control flow" sections, and then go through the files in order. If you need to understand one specific file quickly, use the table of contents.

An important principle worth keeping in mind from the very start: this entire program exists for the sake of a single scientific claim — **"the comparison is fair."** Almost every architectural decision in the code (configuration dataclasses, a fixed seed, result caching, recomputing metrics from history) directly serves that goal. Whenever it is unclear why a piece of code is needed, ask yourself: "which loophole for an unfair comparison does it close?"

The big picture: what scientific problem the program solves

The problem

When a paper says "our optimizer is better than AdamW," that claim is almost always incorrect, because the comparison was carried out carelessly. The classic sources of error are:

1. **An unequal tuning budget.** The new method was tuned for a week, while AdamW was taken with default parameters. A win then only means "we tried harder."

2. **A different task or different data.** It is not the optimizer that changes — everything changes at once.
3. **A single seed.** A difference in the third decimal place turns out to be initialization noise.
4. **Unfair visualization.** "Loss landscape" pictures are drawn along random directions whose scale has nothing to do with the scale of the weights — and from such a picture you can "prove" anything at all.

Choi et al. (2019), "On Empirical Comparisons of Optimizers for Deep Learning", and the AlgoPerf project (Dahl et al.) showed that once these sources of error are removed, the gap between optimizers collapses to almost nothing. Li et al. (2018), "Visualizing the Loss Landscape of Neural Nets", showed the same thing about the pictures: without filter normalization, a landscape slice carries no comparable information.

What this program does

`optbench` is a testbed that carries out such a fair comparison for the scenario of fine-tuning a language model with LoRA, and answers three questions:

RQ1 — quality under an equal budget. Four optimizers (SGD+momentum, AdamW, Lion, Sophia) train the same model on the same data, for the same number of steps, from the same starting point. Each is given the same tuning budget: exactly 18 learning-rate values. The best configuration of each is re-run on 5 seeds, and from those the mean, the standard deviation, and a bootstrap confidence interval are computed.

RQ2 — "tunability." Not "what minimum is reached," but "how wide is the range of learning rates in which the optimizer works at all." This is the more practically important question: an optimizer that gives the same result but only in a narrow LR window is worse in practice.

RQ3 — fair landscape visualization. Two-dimensional slices of the loss function around the solutions that were found are constructed — the correct way (filter normalization) and the naive way — and it is shown that the naive way systematically distorts the picture of the minimum's "sharpness."

The fairness invariant

The project's central rule is stated in a single sentence:

For all optimizers, the following are identical: the task (workload), the budget, and the sweep seed. Only the hyperparameter search space differs.

This rule is not left to the researcher's conscience — it is encoded in the types. In `config/schema.py`, the `ExperimentConfig` object contains **one** `WorkloadConfig` and **one** `BudgetConfig` for the whole experiment, whereas `search_spaces` is a dictionary "optimizer name → its

own space." To give one optimizer a different budget, you would have to rewrite the data structure. This is the technique of "making illegal states unrepresentable."

Repository map

```
optbench/
├── pyproject.toml           package metadata and dependencies
├── requirements-lock.txt    exact versions of all libraries
├── README.md               how to run it
├── src/optbench/           the library (all the logic)
├── scripts/                entry points you launch by hand
├── configs/experiments/    YAML experiment descriptions
├── tests/                  unit and smoke tests (all on CPU)
└── results/                output: tables/*.csv, figures/*.png, runs/*.json
```

Inside `src/optbench/` there are 12 subpackages, each with its own area of responsibility:

Package	Responsible for
<code>config/</code>	experiment description: dataclasses + YAML loader
<code>data/</code>	building datasets and DataLoaders
<code>models/</code>	creating models (toy ones and real HF + LoRA)
<code>optimizers/</code>	the optimizer factory, an in-house Sophia implementation, memory measurement
<code>training/</code>	the training loop and the learning-rate schedule
<code>eval/</code>	computing the loss and evaluating on validation
<code>tuning/</code>	the LR grid, the equal-budget sweep, selecting the best
<code>experiments/</code>	orchestration: sweep → selection → multi-seed run
<code>analysis/</code>	statistics, CSV tables, all plots
<code>landscape/</code>	landscape-visualization primitives (RQ3)
<code>logging/</code>	saving artifacts (JSON/CSV)
<code>reproducibility/</code>	fixing seeds and snapshotting the environment

Plus three files at the package root: `builders.py` (assembling model + data), `run_one.py` (a single training run), `serialize.py` (saving/loading a result).

The separation principle: library vs. scripts

Note an important architectural decision: there is not a single `if __name__ == "__main__"` in `src/`, and there is no logic at all in `scripts/`. The library is a set of pure functions and classes that print nothing to the console on their own (logging is passed in as a

parameter, log=print). The scripts are thin wrappers: parse arguments, load the config, call the library, print the result.

Why this is better: the library can be tested (which is exactly what `tests/` does), reused, and called from a notebook. If the sweep logic lived inside `run_rq1.py`, the test `test_sweep_toy.py` would be impossible.

Control flow: what calls what

Below is the main scenario (`scripts/run_rq1.py`) from launch to writing the tables. An arrow means "calls."

```
scripts/run_rq1.py
├ config/loader.load_experiment(yaml)      -> ExperimentConfig
├ logging/artifacts.ArtifactStore(root)
├ reproducibility/env.capture_env()
├ experiments/rq1_benchmark.run_rq1(cfg, store)
  └ for each optimizer:
      └ tuning/sweep_runner.run_equal_budget_sweep(...)
        └ tuning/search_space.sample_trials(space)  -> [{'lr':...}, ...]
          └ for each trial (if not in cache):
              run_one.train_one(...)
                └ reproducibility/seeding.set_global_seed(seed)
                  └ builders.build_experiment_unit(workload, seed)
                    └ models/hf_causal.load_hf_causal_lm + apply_lora
                      └ data/causal_lm.build_wikitext2_causal_lm
                        └ optimizers/factory.make_optimizer(name, params, lr)
                          └ training/scheduler.build_scheduler(opt, budget)
                            └ training/trainer.Trainer.fit(...)  -> RunResult
                              └ eval/metrics.compute_loss / evaluate
                                └ tuning/selection.select_best_by_val(sweep)  -> best RunResult
                                  └ experiments/multiseed.run_multiseed(...)  -> [RunResult] over 5 seeds
├ analysis/tables.write_tables(out, dir, cfg)  -> 3 CSVs
├ analysis/plots/tunability.plot_tunability(out)  -> PNG
├ analysis/plots/curves.plot_val_curves(out)  -> PNG
```

The `out` structure that `run_rq1` returns and that all the analyzers consume is a dictionary:

```
{
  "adamw": {
    "sweep": [RunResult x 18],  # all sweep trials
    "best": RunResult,          # the best of them
    "seeds": [RunResult x 5],   # re-runs of the best over 5 seeds
  },
  "lion": {...}, "sgd_momentum": {...}, "sophia": {...}
}
```

Memorize this structure — it runs through the entire analytical part of the project. Any function in `analysis/` that takes an `out` argument expects exactly this dictionary.

The key data object: RunResult

Everything that happens in the project ultimately comes down to producing and processing `RunResult` objects (described in `training/trainer.py`). This is the result of a single training run:

```
@dataclass
class RunResult:
    optimizer: str          # optimizer name
    seed: int               # which seed was used
    hparams: dict           # {'lr': 0.003}
    history: list           # [{'step':100,'val_loss':3.4,...}, ...]
    best_val: float         # the best metric value over the run
    best_step: int          # at which step it was reached
    steps_to_target: int | None # when the target threshold was crossed
    tokens_to_target: int | None
    peak_gpu_mem_bytes: int  # peak GPU memory
    mean_step_time_s: float  # mean step time
    final_train_loss: float
    optimizer_state_bytes: int # size of the optimizer state
```

The `history` field is the most important one methodologically: it is exactly what the time-to-target metrics are later recomputed from, which makes it possible to change the threshold without retraining. This is covered in detail in the walkthrough of `analysis/aggregate.py`.

Layer 1. Project packaging

pyproject.toml

Role. The package description for installation, plus tool configuration.

```
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"
[project]
name = "optbench"
version = "0.1.0"
requires-python = ">=3.12"
dependencies = ["torch", "transformers", "peft", "datasets",
               "accelerate", "numpy<2.3", "scipy", "scikit-learn", "matplotlib",
               "pyyaml", "pandas", "lion-pytorch", "tensorboard"]
[tool.setuptools.packages.find]
where = ["src"]
[tool.pytest.ini_options]
testpaths = ["tests"]
addopts = "-q"
```

What matters here.

where `= ["src"]` is the so-called src-layout. The code lives not in `optbench/` at the root, but in `src/optbench/`. Why the extra folder? So that `import optbench` works only after the package has been installed. With the usual layout, Python would pick the folder up from the current directory, and you would be testing something other than what the user will get installed. The src-layout catches mistakes of the "we forgot to add a file to the package" kind.

`numpy<2.3` is the only pinned version among the dependencies. That is a sign that something broke on a newer numpy (usually binary compatibility with torch).

`[tool.pytest.ini_options]` — thanks to this block it is enough to type `pytest` with no arguments and it will find the tests in `tests/`.

Links. The README requires installing the package like this:

```
pip install -r requirements-lock.txt
pip install --no-deps -e .
```

The order is not accidental. `-e` (editable) means "install a link to the sources," so that edits in `src/` are visible immediately without reinstalling. `--no-deps` means "don't touch the dependencies": without this flag pip would see `torch` in the list and might replace the CUDA build installed from `requirements-lock.txt` with a CPU build from PyPI, after which the GPU experiments would stop working.

requirements-lock.txt

The complete list of dependencies with exact versions, including transitive ones (those that were pulled in automatically). This is what makes the results reproducible:

`pyproject.toml` says "torch is needed," while the lock file says "exactly `torch==2.11.0+cu128` is needed."

The README honestly notes a limitation: this torch build is compiled for CUDA 12.8 and a Blackwell-generation GPU. On a different card you need to install your own torch build; nothing else changes.

src/optbench/init.py

```
"""optbench: a fair, reproducible harness for benchmarking and visualizing
optimizers when fine-tuning (small) language models."""
__version__ = "0.1.0"
```

Nothing but the version and a docstring. This is deliberate: a root `__init__.py` that imports half the project would slow down every import and would drag in torch/transformers even where they are not needed.

The other init.py files

Files like `optimizers/__init__.py` act as a "façade" for the subpackage:

```
from optbench.optimizers.factory import HESSIAN_OPTIMIZERS, make_optimizer
from optbench.optimizers.sophia import SophiaG
from optbench.optimizers.state_size import optimizer_state_bytes
__all__ = ["HESSIAN_OPTIMIZERS", "make_optimizer", "SophiaG",
           "optimizer_state_bytes"]
```

That is, you can write `from optbench.optimizers import make_optimizer` instead of the long `from optbench.optimizers.factory import make_optimizer`. `__all__` declares the package's public interface.

Two exceptions:

- `analysis/__init__.py` is empty, just a comment. The reason: importing `matplotlib` is slow, and you do not want to drag it in on every access to `analysis`.
- `landscape/__init__.py` re-exports nothing, but it does contain a project decision in the form of a docstring: RQ3 is deliberately restricted to toy models, because full landscape slices on a real LLM would cost unjustifiably much, whereas the scientific question ("how naive visualization misleads") is demonstrated perfectly well on a small model too. Such "justification comments" occur often in this project — they are worth reading, as they correspond directly to the text of the thesis.

Layer 2. Configuration

This is the layer where the fairness invariant formally lives. Two files: `schema.py` (data structures) and `loader.py` (reading YAML into those structures).

`src/optbench/config/schema.py`

Role. The single source of truth about what an "experiment" is. All nine dataclasses are declared with `@dataclass(frozen=True)` — that is, immutable after creation. An attempt to assign `cfg.budget.max_steps = 100` raises an exception. This is a defence against the most insidious class of bugs: an accidental mutation of the config in the middle of a sweep, because of which half the trials would have run with different settings.

LoraSpec

```
@dataclass(frozen=True)
class LoraSpec:
    enabled: bool = True
    r: int = 8                # rank of the low-rank update
    alpha: int = 16           # scaling factor (usually 2*r)
    dropout: float = 0.05
    target_modules: tuple[str, ...] = ("c_attn",)
    bias: str = "none"
```

A reminder of what LoRA is: instead of training a weight matrix W of size $d \times d$, we freeze it and learn a correction $\Delta W = B \cdot A$, where A has size $r \times d$ and B has size $d \times r$, with r small (8 here). The number of trainable parameters becomes hundreds of times smaller. At inference the result is scaled by α/r .

`target_modules=("c_attn",)` is the name of the module inside GPT-2 where LoRA is inserted. In GPT-2 the `c_attn` layer is one big matrix that produces Q , K and V all at once. In a Llama-style model (SmolLM2) the same projections are stored separately, which is why the second config says `[q_proj, k_proj, v_proj]`. This is precisely the case where "the same task" requires non-identical code for different architectures.

`target_modules` is a tuple, not a list, because a `frozen=True` dataclass must be hashable, and a list is not hashable.

WorkloadConfig — the "unit of work"

```
@dataclass(frozen=True)
class WorkloadConfig:
    name: str
    task: str                # "regression" | "causal_lm"
    model_kind: str          # "toy_mlp" | "toy_transformer" | "hf_causal"
    model_name: str = ""     # HuggingFace id, if model_kind == hf_causal
    dataset: str = ""        # "toy_regression" | "toy_char" | "wikitext2"
    seq_len: int = 128
    micro_batch_size: int = 32
    grad_accum_steps: int = 1
    lora: LoraSpec = field(default_factory=LoraSpec)
    full_finetune: bool = False
    eval_metric: str = "val_loss"
    extras: dict = field(default_factory=dict)
```

The key idea: **a workload is everything that does not depend on the optimizer.** Which model, which data, what batch size, which metric. One such object for the entire experiment — which means all four optimizers physically cannot receive a different task.

Separating `task` and `model_kind` is not obvious, but it is useful: `task` determines how to compute the loss (MSE or cross-entropy), while `model_kind` determines which model to build and how to call it. The toy transformer and GPT-2 solve the same task (`causal_lm`), but are called differently: the first takes a tensor and returns logits, the second takes a dictionary and returns an object with a `.loss` field.

`micro_batch_size × grad_accum_steps` = the effective batch size. The split is needed when the required batch does not fit in GPU memory: we do several small passes and accumulate the gradients.

`extras`: dict is a "pocket" for parameters specific to a particular model or dataset (`d_model`, `hidden`, `max_train_blocks`). It is a compromise: a strict schema for everything that affects the fairness of the comparison, and a free-form dictionary for minor settings. Note that `extras` is the only mutable field in a "frozen" dataclass (`frozen` forbids reassigning a field, but not mutating the dictionary inside it). This is a well-known limitation of `frozen=True`, and here it causes no problems, because nobody modifies `extras`.

BudgetConfig — "how much to compute"

```
@dataclass(frozen=True)
class BudgetConfig:
    kind: str = "steps"          # "steps" | "tokens"
    max_steps: int = 200
    max_tokens: int | None = None
    warmup_ratio: float = 0.0
    eval_every_steps: int = 50
    scheduler: str = "none"      # "none" | "cosine" | "linear"
    grad_clip: float | None = 1.0
```

The second pillar of fairness. All optimizers get exactly `max_steps` steps, the same warmup, the same LR schedule and the same gradient clipping.

Note: `kind="tokens"` and `max_tokens` are declared, but in the current implementation `Trainer.fit` uses only `max_steps`. This is "groundwork for the future" — the field exists in the schema, but there is no logic behind it. You may be asked about this at the defence: the honest answer is "a token budget is provided for by the schema, but the thesis experiments used a step budget."

HParamRange and SearchSpace — the only thing that differs

```
@dataclass(frozen=True)
class HParamRange:
    name: str
    kind: str          # "log_uniform" | "uniform" | "choice" | "fixed"
    low: float | None = None
    high: float | None = None
    choices: tuple = ()
    value: float | None = None

@dataclass(frozen=True)
class SearchSpace:
    optimizer: str
    n_trials: int = 12          # equal tuning budget
    ranges: tuple[HParamRange, ...] = ()
```

`n_trials` is exactly the "equal tuning budget" from the Choi et al. methodology. In the GPT-2 config, all four optimizers have `n_trials: 18`. The ranges, however, are different — and that is correct: a sensible LR for SGD (on the order of 0.1) and for Lion (on the order of $1e-4$) differ by three orders of magnitude. Fairness here means not "the same numbers," but "the same number of attempts and a range declared in advance."

The word "in advance" is essential. A comment in the config says "Pre-declared LR ranges per optimizer (Choi 2019)." If the ranges had been chosen after seeing the result, that would have been fitting to the data.

OptimizerSpec, SeedConfig, RuntimeConfig

```
@dataclass(frozen=True)
class OptimizerSpec:
    name: str          # "sgd_momentum" | "adamw" | "lion" | "sophia"
    defaults: dict = field(default_factory=dict)

@dataclass(frozen=True)
class SeedConfig:
    sweep_seed: int = 0
    final_seeds: tuple[int, ...] = (0, 1, 2, 3, 4)

@dataclass(frozen=True)
class RuntimeConfig:
    device: str = "auto"
    precision: str = "fp32"
    gradient_checkpointing: bool = False
    deterministic: bool = True
    results_root: str = "results/runs"
```

`OptimizerSpec.defaults` — hyperparameters that are not tuned (momenta, weight decay, rho). They are fixed for all trials of a given optimizer.

`SeedConfig` separates two different seeds: `sweep_seed` is the single seed on which the entire LR search runs (otherwise the difference between trials would be a mixture of "the effect of LR" and "the effect of the seed"); `final_seeds` are the K seeds for the final evaluation of the best configuration, from which the spread is computed.

`RuntimeConfig` covers things that ideally should not affect the scientific result: the device, the compute precision, where to store artifacts. The fact that they are separated from `workload` and `budget` is a signal that "this is engineering, not methodology."

ExperimentConfig — everything together

```
@dataclass(frozen=True)
class ExperimentConfig:
    name: str
    rq: str = ""
    workload: "WorkloadConfig | None" = None
    optimizers: tuple[OptimizerSpec, ...] = ()
    search_spaces: dict = field(default_factory=dict) # name -> SearchSpace
    budget: BudgetConfig = field(default_factory=BudgetConfig)
    seeds: SeedConfig = field(default_factory=SeedConfig)
    runtime: RuntimeConfig = field(default_factory=RuntimeConfig)
    target_metric_value: float | None = None
```

Here is where the invariant becomes visible: **one** workload, **one** budget, **one** seeds — while `optimizers` and `search_spaces` are plural. The asymmetry of the structure is the scientific claim.

`target_metric_value` is the threshold for the "steps to target" metric. Why it lives in the config rather than being computed is covered in the walkthrough of `analysis/aggregate.py`.

A comprehension question. Why is `search_spaces` an ordinary `dict` rather than a tuple, when everything else is "frozen"? Answer: a `dict` is not hashable, and it cannot be made an element of a frozen dataclass without the whole object losing hashability. Here strictness was sacrificed for the convenience of access by optimizer name:

```
cfg.search_spaces["adamw"].
```

src/optbench/config/loader.py

Role. Turn a YAML file into a nested structure of dataclasses. Nothing more — no business-logic validation, no default values (those are already in the schema).

```
def load_yaml(path):
    with open(path, "r", encoding="utf-8") as fh:
        return yaml.safe_load(fh)
```

`safe_load` rather than `load` is a matter of principle. Ordinary `yaml.load` is able to construct arbitrary Python objects and therefore allows code to be executed from a YAML file. `safe_load` builds only dictionaries, lists, strings and numbers.

Next comes a set of small private converter functions, one per nesting level:

```
def _lora(d) -> LoraSpec:
    if not d:
        return LoraSpec()
    d = dict(d)
    if "target_modules" in d:
        d["target_modules"] = tuple(d["target_modules"])
    return LoraSpec(**d)
```

Two recurring idioms of the whole file are visible here:

1. `d = dict(d)` — a copy, so as not to spoil the original dictionary that came from YAML;
2. converting lists into tuples — because the dataclasses are frozen.

The most interesting place is the comment in `_range`:

```
def _range(d: dict) -> HParamRange:
    d = dict(d)
    # YAML quirk: "1.0e0" (no sign in exponent) parses as a string, not a float.
    for k in ("low", "high", "value"):
        if d.get(k) is not None:
            d[k] = float(d[k])
    ...
```

This is a real YAML 1.1 trap: the specification requires the exponent to carry a sign. The literal `1.0e-3` parses as a number, whereas `1.0e3` parses as the string `"1.0e3"`. Without this line, the learning rate would sometimes silently become a string, and `np.logspace(log10("1.0e3"))` would crash with an inscrutable error. That is exactly why all the project's configs are written as `1.0e-3`, `1.0e0`, and so on — and the loader still coerces to `float` anyway.

The final assembly is straightforward:

```
def experiment_from_dict(d: dict) -> ExperimentConfig:
    return ExperimentConfig(
        name=d["name"],
        rq=d.get("rq", ""),
        workload=_workload(d["workload"]),
        optimizers=tuple(OptimizerSpec(name=o["name"],
                                      defaults=o.get("defaults", {}))
                      for o in d.get("optimizers", [])),
        search_spaces=_search_spaces(d.get("search_spaces")),
        budget=BudgetConfig(**(d.get("budget") or {})),
        seeds=_seeds(d.get("seeds")),
        runtime=RuntimeConfig(**(d.get("runtime") or {})),
        target_metric_value=d.get("target_metric_value"),
    )
```

Note `d["name"]` and `d["workload"]` without `.get` — these fields are mandatory, and if they are missing the program honestly crashes with a `KeyError` instead of running on defaults. Everything else uses `.get` with the defaults from the schema.

Splitting `experiment_from_dict` from `load_experiment` is also meaningful: the first function is testable without a filesystem, and the second is simply "read the file and call the first."

What to pay attention to. The loader does not check consistency: if `sophia` is listed in `optimizers` but absent from `search_spaces`, the error will surface later, in `run_rq1` at the `cfg.search_spaces[opt_spec.name]` lookup — as a `KeyError`. Explicit validation here would be an improvement.

Layer 3. Experiment descriptions (configs/experiments/)

The YAML files are "data," not "code." There is no logic in them, only values for the schema's fields. Four files form a ladder from the cheapest to the most expensive.

rq1_toy.yaml — a seconds-long pipeline check (CPU)

```
name: rq1_toy
workload: {task: regression, model_kind: toy_mlp, micro_batch_size: 32,
           extras: {in_dim: 8, hidden: 32}}
budget: {max_steps: 80, eval_every_steps: 40, scheduler: none}
optimizers: [adamw, lion]
search_spaces: n_trials: 4 for each
seeds: {sweep_seed: 0, final_seeds: [0, 1]}
target_metric_value: null
runtime: {device: cpu, precision: fp32}
```

The task is a synthetic linear regression, and the model is a two-layer MLP. The point of this config is not a scientific result, but to run the whole path in a few seconds: sweep → selection → multi-seed run → tables → plots. If something broke in the analytics, you will find out here rather than after three hours of GPU time.

`target_metric_value: null` — there is no threshold, so in the resulting table

`rq1_toy_rq1_main.csv` the `steps_to_target_mean` column is empty. That is expected, not a bug.

rq1_gpt2_smoke.yaml — a check of the GPT-2 path (GPU, minutes)

The same pipeline, but now on a real GPT-2 with LoRA: `max_steps: 40`, 2 trials, 2 seeds, `max_train_blocks: 64`. The goal is to make sure that loading the HF model, the LoRA wrapper, bf16 and working with CUDA do not crash, before launching a multi-hour run.

This is a very practical technique: between "works on the toy" and "works on the GPU" lies a whole class of errors (out-of-memory, dtype, the dataset missing from the cache), and it is cheaper to catch them in 40 steps.

rq1_gpt2_lora.yaml — the main experiment

The main file of the entire work. Let us go through it block by block.

```
workload:
  model_kind: hf_causal
  model_name: gpt2
  dataset: wikitext2
  seq_len: 256
  micro_batch_size: 8
  lora: {enabled: true, r: 8, alpha: 16, dropout: 0.05,
        target_modules: [c_attn]}
  extras: {max_train_blocks: 2500, max_eval_blocks: 128}
```

`max_train_blocks: 2500` at `seq_len: 256` means 640,000 training tokens. Truncating the corpus is a deliberate compromise, recorded in a comment in `data/causal_lm.py`: so that one trial fits into a few minutes. And there are $4 \times 18 = 72$ trials, plus $4 \times 5 = 20$ final runs.

```
budget:
  max_steps: 800
  eval_every_steps: 100
  warmup_ratio: 0.05      # 40 warmup steps
  scheduler: cosine
  grad_clip: 1.0
```

The same for everyone. $800 \text{ steps} \times 8 \text{ examples} \times 256 \text{ tokens} \approx 1.64 \text{ M tokens per run}$ — that is, the model sees the corpus roughly 2.5 times.

```
optimizers:
  - {name: sgd_momentum, defaults: {momentum: 0.9, nesterov: true}}
  - {name: adamw,          defaults: {betas: [0.9, 0.999], weight_decay: 0.01}}
  - {name: lion,           defaults: {betas: [0.9, 0.99], weight_decay: 0.0}}
  - {name: sophia,         defaults: {rho: 0.04, betas: [0.965, 0.99]}}
```

The choice of optimizers is not accidental; it builds a hierarchy of nesting:

SGD+momentum is the basic first-order method; **AdamW** is the same thing plus a per-coordinate adaptive scale; **Lion** is a sign-based update whose state is half the size of AdamW's; **Sophia** is a method that uses a curvature estimate (the Hessian diagonal). That is, what is being compared is not four random methods but a "ladder" from simple to complex.

```
search_spaces:
  sgd_momentum: n_trials 18, lr from 1e-3 to 1.0
  adamw:        n_trials 18, lr from 1e-5 to 1e-2
  lion:         n_trials 18, lr from 1e-6 to 1e-3
  sophia:       n_trials 18, lr from 1e-5 to 1e-2
```

Three decades each, 18 points on a logarithmic grid — that is, a step of roughly $1.5\times$. The ranges are shifted in accordance with known practice: Lion requires an LR about $10\times$ smaller than AdamW, while SGD requires one 2–3 orders of magnitude larger.

```
target_metric_value: 3.20
```

The comment above this line is a model of scientific honesty and is worth reading in full:

Time-to-target threshold (val_loss), AlgoPerf-style: a small margin above the slowest optimizer's converged floor (Lion ~3.188), so every optimizer's best config reaches it across all seeds and the steps-to-target ranking is meaningful. (3.36 was too easy: all best configs crossed it at the first eval, step 100.)

That is: the threshold was chosen not arbitrarily but by a rule — "slightly above the level reached by the weakest participant." If the threshold is too easy, everyone crosses it at the very first evaluation and the metric stops discriminating. If it is too strict, some optimizers never reach it at all and there is nothing to average over. The first unsuccessful attempt (3.36) was not hidden in the comment but recorded.

rq1b_smollm2_lora.yaml — a transferability check

The second config checks whether the ranking of optimizers transfers to another model. GPT-2 is a GPT-style architecture (LayerNorm, learned positional embeddings); SmolLM2-360M is Llama-style (RoPE, SwiGLU, RMSNorm). The comment states the scientific position in advance:

The question is the RANKING, not a new absolute number — either outcome is a result: holds -> protocol transfers; breaks -> AlgoPerf's "one task is not enough" shown firsthand.

This is an important methodological move: **both outcomes are declared to be a result before the experiment.** That eliminates the temptation to fit the interpretation to whatever came out.

The budget is lightened: 500 steps, 7 LR points instead of 18, 3 seeds instead of 5. The ranges are narrowed and centred on the optimum found on GPT-2 (± 1 order of magnitude). The threshold is recomputed by the same rule: 2.48, with SGD's "floor" around 2.477.

A limitation you need to know about. With 3 seeds against 3, the minimum attainable p-value of the Mann-Whitney test is 0.1. That is why in the file `rq1b_smollm2_lora_rq1_significance.csv` all three rows show exactly $p = 0.1$ — this is not "almost significant," this is the floor of the statistic. More on this in the section on `analysis/aggregate.py`.

Layer 4. Data

src/optbench/data/base.py

The whole file is 15 lines:

```
@dataclass
class DataModule:
    task: str          # "regression" | "causal_lm"
    train_loader: object
    val_loader: object
    test_loader: object
    meta: dict = field(default_factory=dict)
```

A simple container: three loaders, the task type and a metadata dictionary. Why is it needed if you could just return a tuple of three loaders? Because `meta` carries information that is needed further down the pipeline: `vocab_size` (to build a model of the right size), `in_dim`, `model_kind`, `grad_accum_steps`. The trainer reads `data.meta["model_kind"]` rather than receiving it as a separate argument — the data and the model travel together.

Note: this dataclass is not frozen, because `builders.py` adds two more fields to `meta` after creation.

src/optbench/data/toy.py

Role. Synthetic datasets for the tests and for RQ3.

```
_CORPUS = "the quick brown fox jumps over the lazy dog. " * 200
```

A small repeating corpus. For checking "does the loss go down" this is enough, and training takes seconds.

```
class _DictDataset(Dataset):
    def __init__(self, tensors: dict):
        self.tensors = tensors
        self.n = len(next(iter(tensors.values())))
    def __len__(self): return self.n
    def __getitem__(self, i): return {k: v[i] for k, v in self.tensors.items()}
```

The dataset yields a dictionary, not a tuple. This is a key convention of the whole project: **a batch is always a dictionary**. Thanks to that, the line in the trainer

```
batch = {k: v.to(device) for k, v in next(data_iter).items()}
```

works identically for regression (`{"x", "y"}`), for the toy LM (`{"input_ids", "labels"}`), and for HuggingFace (`{"input_ids", "attention_mask", "labels"}`). PyTorch's standard `default_collate` knows how to assemble dictionaries into dictionary-batches by itself.

```
def _loaders(train, val, test, batch_size, seed):
    g = torch.Generator().manual_seed(seed)
    train_loader = DataLoader(train, batch_size=batch_size, shuffle=True,
                              generator=g, drop_last=True)
    val_loader = DataLoader(val, batch_size=batch_size, shuffle=False)
    ...
```

Three details, each about reproducibility and fairness:

- `generator=g` with an explicit seed: the shuffling order is fixed. Without this, two optimizers would see the data in different orders, and part of the difference in results would be explained by that.
- `drop_last=True`: the last incomplete batch is discarded so that all steps are of the same size.
- `shuffle=False` for validation: the evaluation must be deterministic.

The regression generator:

```
def build_toy_regression(in_dim=8, n=900, batch_size=32, seed=0, noise=0.1):
    rng = np.random.default_rng(seed)
    x = rng.standard_normal((n, in_dim)).astype("float32")
    w = rng.standard_normal((in_dim, 1)).astype("float32")
    y = (x @ w + noise * rng.standard_normal((n, 1))).astype("float32")
```

A classic linear problem with noise: $y = Xw + \epsilon$. Its exact solution is known, the loss is guaranteed to go down, and the minimum attainable MSE is roughly equal to the noise variance (0.01). The split is 70/15/15 into train/val/test.

The character-LM generator:

```
seqs = [ids[i : i + seq_len + 1]
         for i in range(0, len(ids) - seq_len - 1, seq_len)]
inp = torch.from_numpy(seqs[:, :-1])
lab = torch.from_numpy(seqs[:, 1:])
```

Slicing into non-overlapping windows of length `seq_len+1`, then a shift by one character: the input is all characters except the last, the target is all except the first. That is exactly the "predict the next token" setup. The shift is done here, by hand — unlike the HF path, where the model shifts the labels internally. It is important to remember the difference; it explains why there are two different branches in `eval/metrics.py`.

src/optbench/data/causal_lm.py

Role. Real data: WikiText-2, tokenization and packing into blocks.

```
class _BlockDataset(Dataset):
    def __getitem__(self, i):
        ids = self.input_ids[i]
        return {"input_ids": ids,
                "attention_mask": torch.ones_like(ids),
                "labels": ids.clone()}
```

`labels = input_ids` is not a mistake. HuggingFace causal-LM models shift the labels by one token inside `forward` themselves, so the same ids must be supplied. `attention_mask` of all ones — because there is no padding at all.

```
def _tokenize_and_block(texts, tokenizer, seq_len, max_blocks=None):
    joined = "\n\n".join(t for t in texts if t and t.strip())
    ids = tokenizer(joined, return_attention_mask=False)["input_ids"]
    n_blocks = len(ids) // seq_len
    if max_blocks is not None:
        n_blocks = min(n_blocks, max_blocks)
    ids = ids[: n_blocks * seq_len]
    return torch.tensor(ids, dtype=torch.long).view(n_blocks, seq_len)
```

This is packing — the standard technique for language modelling. Instead of padding short strings (and wasting compute), the whole corpus is glued into one long token sequence and cut into chunks of exactly `seq_len`. The upsides: zero padding, all batches of the same shape, and a strictly constant number of tokens per step — which matters for the "equal budget." The downside: a block may start and end in the middle of a sentence. For perplexity evaluation this is common practice.

```
# "wikitext" (legacy script dataset) no longer resolves in datasets >=4;
# the canonical parquet home is "Salesforce/wikitext".
ds = load_dataset("Salesforce/wikitext", "wikitext-2-raw-v1")
```

An example of a useful comment about an infrastructure breakage: in newer versions of the `datasets` library the old name stopped working. Without such a comment, the next person will spend half an hour looking for the cause.

A subtlety about `max_train_blocks`. The corpus is truncated before the `DataLoader` is created, so all optimizers see the same prefix of the data. If the truncation were random and unseeded, that would become a source of unfairness.

Layer 5. Models

src/optbench/models/toy_mlp.py

```
class ToyMLP(nn.Module):
    def __init__(self, in_dim=8, hidden=32, out_dim=1, depth=2):
        super().__init__()
        layers = []
        d = in_dim
        for _ in range(depth):
            layers += [nn.Linear(d, hidden), nn.GELU()]
            d = hidden
        layers += [nn.Linear(d, out_dim)]
        self.net = nn.Sequential(*layers)
```

The simplest model in the project: a few linear layers with GELU. Used in the tests and in the pipeline check. The docstring mentions "full-Hessian curvature ground truth" — the model is small enough to compute the full Hessian and compare it with the cheap diagonal approximation that Sophia uses. In the final version of the code that check is absent, but the model was designed with it in mind.

src/optbench/models/toy_transformer.py

```
class ToyCharTransformer(nn.Module):
    def __init__(self, vocab_size, d_model=64, nhead=2, num_layers=2,
                 dim_ff=128, max_len=256, dropout=0.0):
        self.tok = nn.Embedding(vocab_size, d_model)
        self.pos = nn.Embedding(max_len, d_model)
        layer = nn.TransformerEncoderLayer(d_model, nhead, dim_ff, dropout,
                                           batch_first=True, activation="gelu")
        self.encoder = nn.TransformerEncoder(layer, num_layers)
        self.ln = nn.LayerNorm(d_model)
        self.head = nn.Linear(d_model, vocab_size)
```

A minimal decoder transformer: token embeddings + learned positional embeddings → a few self-attention layers → normalization → projection into the vocabulary.

```
def forward(self, idx):
    _, t = idx.shape
    pos = torch.arange(t, device=idx.device)
    x = self.tok(idx) + self.pos(pos)[None, :, :]
    mask = torch.triu(torch.full((t, t), float("-inf")),
                      device=idx.device), diagonal=1)
    x = self.encoder(x, mask=mask)
    return self.head(self.ln(x))
```

The most essential thing here is the **causal mask**. `torch.triu(..., diagonal=1)` builds an upper-triangular matrix of `-inf` above the main diagonal. After the softmax these positions receive zero weight, i.e. position i cannot "peek at" tokens $j > i$. Without this

mask the model would solve the task trivially by copying the answer from the future, and the loss would drop to almost zero — a typical mistake in educational implementations.

Formally a `TransformerEncoderLayer` (an encoder) is used, but with a causal mask it behaves like a decoder. This is a common technique.

`dropout=0.0` by default: RQ3 needs a deterministic loss function, otherwise the landscape surface would be noised by dropout randomness.

src/optbench/models/hf_causal.py

Role. Loading a real model from HuggingFace and wrapping it in LoRA.

```
def load_hf_causal_lm(model_name, *, gradient_checkpointing=False):
    from transformers import AutoModelForCausalLM, AutoTokenizer
    tokenizer = AutoTokenizer.from_pretrained(model_name)
    if tokenizer.pad_token is None:
        tokenizer.pad_token = tokenizer.eos_token
    model = AutoModelForCausalLM.from_pretrained(model_name)
    if hasattr(model.config, "use_cache"):
        model.config.use_cache = False
    if gradient_checkpointing:
        model.gradient_checkpointing_enable()
        model.enable_input_require_grads()
    return model, tokenizer
```

A line-by-line walkthrough:

- **The import inside the function.** `transformers` is imported lazily so that the toy path and the tests do not pay seconds of startup for it. The same technique appears in `builders.py` and `data/causal_lm.py`.
- `pad_token = eos_token`: GPT-2 has no padding token. Here it is not actually needed (there is no padding, thanks to packing), but a tokenizer without a `pad_token` emits warnings and crashes in some code paths.
- `use_cache = False`: the KV cache is needed for generation, but during training it only eats memory and conflicts with gradient checkpointing.
- `enable_input_require_grads()`: a LoRA + checkpointing subtlety. The base weights are frozen, so a block's input does not require a gradient, and checkpointing "breaks" the graph. This call forcibly turns on `requires_grad` for the input embeddings.

```
def apply_lora(model, lora_spec):
    from peft import LoraConfig, get_peft_model
    config = LoraConfig(r=lora_spec.r, lora_alpha=lora_spec.alpha,
                        lora_dropout=lora_spec.dropout,
                        target_modules=list(lora_spec.target_modules),
                        bias=lora_spec.bias, task_type="CAUSAL_LM")
    return get_peft_model(model, config)
```

`get_peft_model` returns a wrapper in which the base weights are frozen (`requires_grad=False`) and only the matrices A and B remain trainable. For GPT-2 with `r=8` on `c_attn` that is on the order of 0.3 % of the model's parameters.

That is exactly why, later on in `run_one.py`, the optimizer is given not all the parameters but a filtered set:

```
def _trainable(model):
    return [p for p in model.parameters() if p.requires_grad]
```

Had all of them been passed, AdamW would have allocated momentum buffers for 124 million frozen parameters and wasted gigabytes.

```
def count_parameters(model) -> tuple[int, int]:
    trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)
    total = sum(p.numel() for p in model.parameters())
    return trainable, total
```

Used by the scripts to print a line like "trainable 294,912 / 124,734,720 (0.236 %)".

The docstring at the top of the file explains the precision choice: the base weights are kept in fp32, and bf16 is enabled only as an autocast for the duration of the computation. That way the optimizer state and the gradients remain in fp32 — which matters for numerical stability and for curvature analysis.

src/optbench/builders.py

Role. The single point where a concrete model and concrete data are created from a task description.

```
def build_experiment_unit(workload, seed=0):
    if workload.model_kind == "hf_causal":
        return _build_hf_unit(workload, seed)
    data = build_data(workload, seed)
    model = build_model(workload, data.meta, seed)
    return model, data
```

A dispatcher on `model_kind`. Note the order in the toy branch: **first the data, then the model**, because the model needs `data.meta["vocab_size"]` or `in_dim`. The data dictates the shape of the model, not the other way round.

```
def build_data(workload, seed=0):
    if workload.model_kind == "toy_mlp" or workload.dataset == "toy_regression":
        dm = build_toy_regression(...)
    elif workload.model_kind == "toy_transformer" or workload.dataset == "toy_char":
        dm = build_toy_char_lm(...)
    else:
        raise ValueError(f"no toy data builder for workload: {workload.name}")
    dm.meta["model_kind"] = workload.model_kind
    dm.meta["grad_accum_steps"] = workload.grad_accum_steps
    return dm
```

The last two lines are exactly the additions to meta for whose sake `DataModule` is not frozen. The trainer later reads these fields from the data.

The HuggingFace branch:

```
def _build_hf_unit(workload, seed):
    from optbench.data.causal_lm import build_wikitext2_causal_lm
    from optbench.models.hf_causal import apply_lora, load_hf_causal_lm
    model, tokenizer = load_hf_causal_lm(workload.model_name, ...)
    if workload.lora.enabled and not workload.full_finetune:
        model = apply_lora(model, workload.lora)
    if workload.dataset in ("wikitext2", "wikitext-2", ""):
        data = build_wikitext2_causal_lm(tokenizer, seq_len=..., seed=seed, ...)
    else:
        raise ValueError(...)
    data.meta["model_kind"] = "hf_causal"
    data.meta["grad_accum_steps"] = workload.grad_accum_steps
    return model, data
```

Here the order is reversed: **first the model, then the data**, because the data is tokenized with the model's tokenizer. This is an important dependency: with GPT-2 the corpus is cut up by GPT-2's BPE vocabulary, and with SmolLM2 by its own. That means the `val_loss` of the two models is not directly comparable — they have different vocabularies and a different "difficulty" per token. The work takes this into account: in the second experiment it is not the absolute number that is compared but the ordering of the optimizers.

All the `raise ValueError`s in this file are examples of the "fail loudly" principle. If you ask for an unsupported dataset, the program stops with a clear message instead of silently substituting something else.

Layer 6. Optimizers

`src/optbench/optimizers/factory.py`

Role. A single function that builds any of the four optimizers from a string name.

```

HESSIAN_OPTIMIZERS = {"sophia"}

def make_optimizer(name: str, params, lr: float, **kwargs):
    key = name.lower()
    if key in ("sgd_momentum", "sgd"):
        return torch.optim.SGD(params, lr=lr,
                                momentum=kwargs.get("momentum", 0.9),
                                nesterov=kwargs.get("nesterov", True),
                                weight_decay=kwargs.get("weight_decay", 0.0))
    if key == "adamw":
        return torch.optim.AdamW(params, lr=lr,
                                   betas=tuple(kwargs.get("betas", (0.9, 0.99))),
                                   eps=kwargs.get("eps", 1e-8),
                                   weight_decay=kwargs.get("weight_decay", 0.01))
    if key == "lion":
        from lion_pytorch import Lion
        return Lion(params, lr=lr,
                     betas=tuple(kwargs.get("betas", (0.9, 0.99))),
                     weight_decay=kwargs.get("weight_decay", 0.0))
    if key == "sophia":
        return SophiaG(params, lr=lr,
                         betas=tuple(kwargs.get("betas", (0.965, 0.99))),
                         rho=kwargs.get("rho", 0.04),
                         weight_decay=kwargs.get("weight_decay", 0.1))
    raise ValueError(f"unknown optimizer: {name}")

```

This is the classic factory pattern. Why it is needed: the sweep must be able to build any optimizer knowing only a string from the YAML. Without the factory you would have to build a chain of ifs in `run_one.py`, and the configs would stop being purely declarative.

`tuple(kwargs.get("betas", ...))` is a forced conversion to a tuple, because from YAML `betas` arrives as a list while PyTorch expects a tuple.

`HESSIAN_OPTIMIZERS = {"sophia"}` is the set of optimizers that need a periodic curvature recomputation. In the trainer itself, however, it is not this set that is used but a check via `hasattr(optimizer, "update_hessian")` — "duck typing." The constant remains as documentation of intent.

A subtlety worth knowing about. The default `weight_decay` values differ across optimizers: SGD — 0.0, AdamW — 0.01, Lion — 0.0, Sophia — 0.1. In the config `rq1_gpt2_lora.yaml` no weight decay is set explicitly for Sophia, so the default of 0.1 applies — ten times larger than AdamW's. Formally the fairness invariant is not violated (the non-tuned hyperparameters are fixed for all trials of a given optimizer and are taken from the method authors' recommendations), but this is an asymmetry worth being aware of and worth being able to explain. If you wanted to remove it, it would be enough to add `weight_decay: 0.01` to Sophia's defaults.

src/optbench/optimizers/sophia.py

Role. An in-house implementation of the Sophia optimizer (Liu et al., 2024). It is the only optimizer written from scratch in this project — so we will go through it in detail.

The idea of the method

AdamW divides the gradient by the square root of the mean squared gradient — that is an adaptation to the scale of the gradient. Sophia divides by a curvature estimate (the Hessian diagonal) — an adaptation to the curvature of the loss function. The intuition: along directions where the surface bends steeply the step should be small; along flat ones, large.

Two features make the method practical:

1. The curvature is recomputed not every step but once every k steps (here $k=10$) — otherwise it would be too expensive.
2. The update is clipped coordinate-wise: even if the curvature estimate turned out to be bad, no parameter moves by more than lr .

The constructor

```
class SophiaG(Optimizer):
    def __init__(self, params, lr=2e-4, betas=(0.965, 0.99), rho=0.04,
                 weight_decay=1e-1, eps=1e-12):
        if lr <= 0:
            raise ValueError(f"invalid lr: {lr}")
        defaults = dict(lr=lr, betas=betas, rho=rho,
                       weight_decay=weight_decay, eps=eps)
        super().__init__(params, defaults)
```

Inheriting from `torch.optim.Optimizer` gives you for free: parameter groups (`param_groups`), a state dictionary (`self.state`), the `zero_grad()` method, and compatibility with `LambdaLR`. That is exactly why Sophia can be dropped into the trainer in place of AdamW without a single edit.

`betas[0] = 0.965` is the momentum EMA coefficient; `betas[1] = 0.99` is the curvature EMA coefficient (slower, because curvature changes more smoothly). `rho = 0.04` is the parameter that sets the clipping threshold.

The curvature update

```
@torch.no_grad()
def update_hessian(self) -> None:
    for group in self.param_groups:
        beta2 = group["betas"][1]
        for p in group["params"]:
            if p.grad is None:
                continue
            state = self.state.setdefault(p, {})
            if "hessian" not in state:
                state["hessian"] = torch.zeros_like(p)
            state["hessian"].mul_(beta2).addcmul_(p.grad, p.grad, value=1 - beta2)
```

The line `state["hessian"].mul_(beta2).addcmul_(p.grad, p.grad, value=1 - beta2)` implements an exponential moving average of squared gradients:

$$h \leftarrow \beta_2 \cdot h + (1 - \beta_2) \cdot g \odot g$$

`addcmul_` is the operation "add the element-wise product of two tensors with a coefficient," performed in place, without allocating new memory. In PyTorch the `_` suffix always means in-place.

Here lies **the main departure from the original paper**, and it is honestly documented in the file's docstring:

The original uses a Gauss-Newton-Bartlett estimator requiring an extra sampled backward. Here we use the diagonal of the empirical Fisher (an EMA of `grad*grad`) computed from the gradients the trainer already produces. This keeps Sophia a pure optimizer with no extra forward/backward; the simplification is stated explicitly in the thesis methodology.

What this means. The original Sophia estimates the Hessian diagonal via a Gauss-Newton-Bartlett estimator: you have to sample labels from the model's distribution and do an extra backward pass. Here, instead, the diagonal of the empirical Fisher matrix is taken — the mean of the squares of the gradients that have already been computed.

Why this is admissible: for log-likelihood problems (and language modelling is exactly that), the Fisher matrix coincides with the Hessian at the optimum and is a positive-definite approximation to it away from the optimum. The practical difference: the empirical Fisher uses the true labels rather than sampled ones, and is therefore biased — near the minimum it underestimates the curvature.

Why this simplification was chosen: so that Sophia remains "just an optimizer" that needs no access to the model and the data. An extra backward would have required changing the `Trainer.fit` interface specially for one method — and then the step-time comparison would have become entirely incommensurable.

This is the point you will almost certainly be asked about at the defence. The correct answer: "We implemented Sophia-G with the empirical-Fisher diagonal instead of the GNB estimator, because the latter requires an extra backward pass and would have broken the uniformity of the interface; the simplification is recorded in the methodology text and means that our conclusions apply to this variant of the method rather than to the authors' exact publication."

The optimizer step

```
@torch.no_grad()
def step(self, closure=None):
    loss = None
    if closure is not None:
        with torch.enable_grad():
            loss = closure()
    for group in self.param_groups:
        beta1 = group["betas"][0]
        lr, rho, wd, eps = (group["lr"], group["rho"],
                             group["weight_decay"], group["eps"])
        for p in group["params"]:
            if p.grad is None:
                continue
            grad = p.grad
            state = self.state.setdefault(p, {})
            if "exp_avg" not in state:
                state["exp_avg"] = torch.zeros_like(p)
            if "hessian" not in state:
                state["hessian"] = torch.zeros_like(p)
            exp_avg, hess = state["exp_avg"], state["hessian"]
            exp_avg.mul_(beta1).add_(grad, alpha=1 - beta1)
            if wd != 0:
                p.mul_(1 - lr * wd)
            denom = (rho * hess).clamp(min=eps)
            ratio = (exp_avg.abs() / denom).clamp(max=1.0)
            p.addcmul_(exp_avg.sign(), ratio, value=-lr)
    return loss
```

Let us go through the formula line by line.

1. Momentum. `exp_avg.mul_(beta1).add_(grad, alpha=1 - beta1)` is

$$m \leftarrow \beta_1 \cdot m + (1 - \beta_1) \cdot g$$

ordinary exponential smoothing of the gradient.

2. Decoupled weight decay. `p.mul_(1 - lr * wd)` is

$$\theta \leftarrow \theta \cdot (1 - lr \cdot \lambda)$$

Multiplying the weights by a number slightly below one before the main update. This is the "decoupled weight decay" from AdamW: the decay is not mixed into the gradient but applied to the weights directly, so it is not distorted by the adaptive scale.

3. The update.

```
denom = max( $\rho \cdot h$ ,  $\epsilon$ ) ratio = min(|m| / denom, 1)  $\theta \leftarrow \theta - lr \cdot \text{sign}(m) \cdot \text{ratio}$ 
```

This is the most substantive line. Let us examine two limiting cases:

- If $|m| \geq \rho \cdot h$ (the momentum is large compared with the curvature), then $\text{ratio} = 1$ and the update equals $-lr \cdot \text{sign}(m)$. That is, a step of fixed length `lr` in the descent direction — exactly as in Lion.
- If $|m| < \rho \cdot h$ (the curvature is large), then $\text{ratio} < 1$ and the step equals $-lr \cdot m / (\rho \cdot h)$ — a Newton step divided by the curvature.

Hence the meaning of `rho`: it sets the boundary between these two regimes. A small `rho` → clipping triggers more often → the method is closer to Lion. A large `rho` → the Newton regime more often.

`clamp(min=eps)` with `eps=1e-12` protects against division by zero: at the very start of training hessian is a zero tensor (`update_hessian` has not been called yet), so `denom` would be zero. With the guard, `ratio` is clipped to 1, and for the first 10 steps Sophia behaves as a sign-based method. This is sensible "cold start" behaviour.

Why there is no bias correction. In Adam there is a division by $1 - \beta_1^t$, because the EMA starts from zero and the first steps are underestimated. Here there is no correction — and there is none in the original Sophia either, since the `sign(m)` operation is insensitive to scaling `m` by a positive number, and the `clamp(max=1)` clipping keeps the influence of the underestimate bounded.

src/optbench/optimizers/state_size.py

```
def optimizer_state_bytes(optimizer) -> int:
    total = 0
    for state in optimizer.state.values():
        for value in state.values():
            if torch.is_tensor(value):
                total += value.numel() * value.element_size()
    return total
```

Eight lines, but this is one of the axes of comparison in the work. The function walks the optimizer's state dictionary (`self.state` is `param → {'exp_avg': tensor, ...}`) and sums the bytes of all tensors. The `torch.is_tensor` check is needed because the state sometimes contains scalars too (for example Adam's step counter).

What is expected theoretically and what came out in the results (for LoRA GPT-2):

Optimizer	Stores	Expected	Actual, MB
SGD+momentum	momentum	1×	1.18
Lion	momentum	1×	1.18
AdamW	momentum + variance	2×	2.36
Sophia	momentum + curvature	2×	2.36

The agreement with theory is perfect — a good "sanity" indication that both the Sophia implementation and the measurement are correct.

A subtlety. The function is called at the end of `Trainer.fit`. If it were called before the first step it would return 0: PyTorch creates state buffers lazily, on the first `step()`. The test `test_optimizers.py` checks for `> 0` precisely after training.

Layer 7. Training

src/optbench/training/scheduler.py

```
def build_scheduler(optimizer, budget):
    if budget.scheduler == "none":
        return None
    total = max(1, budget.max_steps)
    warmup = int(budget.warmup_ratio * total)
    def lr_lambda(step: int) -> float:
        if warmup > 0 and step < warmup:
            return step / max(1, warmup)
        progress = (step - warmup) / max(1, total - warmup)
        if budget.scheduler == "cosine":
            return 0.5 * (1.0 + math.cos(math.pi * min(progress, 1.0)))
        if budget.scheduler == "linear":
            return max(0.0, 1.0 - progress)
        return 1.0
    return LambdaLR(optimizer, lr_lambda)
```

`LambdaLR` multiplies the base learning rate by the value of `lr_lambda(step)`. What is implemented here is the standard "warmup + cosine" schedule:

- **Warmup** (the first 5 % of steps = 40 out of 800): the multiplier grows linearly from 0 to 1. It is needed because adaptive methods have poor moment estimates at the start of training and can take a destructively large step.
- **Cosine decay**: the multiplier goes smoothly from 1 to 0 according to $0.5 \cdot (1 + \cos(\pi \cdot \text{progress}))$. By the end of the budget the LR reaches zero, which helps to "settle" into the minimum.

It is important that the schedule is built from `budget`, which is shared by all optimizers. That is, the shape of the LR curve is identical; only the base value differs.

A subtlety worth knowing about. On creation, LambdaLR immediately calls `lr_lambda(0)`. With `warmup > 0` this gives a multiplier of 0, i.e. the optimizer's first step is taken with a zero learning rate and effectively changes nothing. The effect is the same for all optimizers, so fairness does not suffer, but formally the budget is 799 "working" steps, not 800.

The call order in the trainer is also standard: first `optimizer.step()`, then `scheduler.step()`. The reverse order would trigger a PyTorch warning and shift the schedule by one step.

src/optbench/training/trainer.py

Role. The core of the whole project. This is where the fairness invariant is physically enforced: the same loop, the same budget, the same evaluation schedule — for any optimizer.

Helper functions

```
def resolve_device(spec: str) -> torch.device:
    if spec == "cpu": return torch.device("cpu")
    if spec == "cuda": return torch.device("cuda")
    return torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

"auto" — pick the GPU if there is one. Explicit "cpu"/"cuda" — do not guess. This lets the tests demand CPU rigidly and get reproducibility.

```
def _autocast_factory(device, precision):
    if device.type == "cuda" and precision in ("bf16", "fp16"):
        dtype = torch.bfloat16 if precision == "bf16" else torch.float16
        return lambda: torch.autocast(device_type="cuda", dtype=dtype)
    return None
```

What is returned is a factory of context managers, not a manager itself — because `torch.autocast` is single-use and each pass needs a fresh one. On CPU or in fp32, `None` is returned and the trainer substitutes `nullcontext()`.

What autocast buys you: matrix multiplications are performed in bf16 (twice as fast and half the memory), while the weights, gradients and optimizer state stay in fp32. Unlike fp16, bf16 has the same exponent range as fp32 — so no `GradScaler` is needed and no overflows occur.

```
def _infinite(loader):
    while True:
        for batch in loader:
            yield batch
```

A generator that cycles the loader endlessly. Thanks to it the training loop counts **steps**, **not epochs**. This is directly required by the methodology: the "equal budget" is specified

in steps, and epochs simply have nothing to do with it. When the data runs out, the `DataLoader` reshuffles (continuing the state of the same generator — i.e. deterministically).

The constructor

```
class Trainer:
    def __init__(self, model, data, *, device="auto", precision="fp32",
                 grad_clip=1.0):
        self.device = resolve_device(device)
        self.model = model.to(self.device)
        self.data = data
        self.task = data.task
        self.model_kind = data.meta["model_kind"]
        self.grad_accum = int(data.meta.get("grad_accum_steps", 1))
        self.grad_clip = grad_clip
        self.precision = precision
        self.autocast = _autocast_factory(self.device, precision)
```

The trainer pulls `task`, `model_kind` and `grad_accum_steps` out of the data rather than out of the config. That is why it can be created given only `(model, data)` — which is exactly what the tests and `run_landscape.py` do, without assembling a full `ExperimentConfig`.

The `*` in the signature makes all subsequent arguments keyword-only: `Trainer(m, d, "cuda")` will not work, you must write `device="cuda"`. A guard against confusion in argument order.

The main loop

```
def fit(self, optimizer, budget, *, target_value=None,
       eval_metric="val_loss", scheduler=None, opt_name="opt", seed=0,
       hparams=None, logger=None, hessian_update_interval=10) -> RunResult:
    device, model = self.device, self.model
    if device.type == "cuda":
        torch.cuda.reset_peak_memory_stats(device)
    history = []
    best_val, best_step = float("inf"), -1
    steps_to_target = tokens_to_target = None
    tokens_seen = 0
    step_times = []
    last_loss = float("nan")
    data_iter = _infinite(self.data.train_loader)
    model.train()
```

`reset_peak_memory_stats` zeroes CUDA's peak-memory counter — otherwise the measurement would include memory occupied by a previous run in the same process.

`best_val = float("inf")` — the metric is always minimized (it is a loss), hence starting from infinity.

The body of the loop:

```

for step in range(1, budget.max_steps + 1):
    t0 = time.perf_counter()
    optimizer.zero_grad(set_to_none=True)
    for _ in range(self.grad_accum):
        batch = {k: v.to(device) for k, v in next(data_iter).items()}
        ctx = self.autocast() if self.autocast else nullcontext()
        with ctx:
            loss, ntok = compute_loss(model, batch, self.task, self.model_kind)
            (loss / self.grad_accum).backward()
            last_loss = float(loss.item())
            tokens_seen += ntok

```

The key points:

- `zero_grad(set_to_none=True)` — zeroing gradients by replacing them with `None` instead of writing zeros. Saves memory and is slightly faster.
- The inner loop over `grad_accum`: several small passes, with gradients accumulating by themselves (PyTorch sums them at each `backward`).
- `(loss / self.grad_accum).backward()` — the division is mandatory: otherwise the sum of gradients over `k` micro-batches would be `k` times larger than the gradient of one big batch, and the effective learning rate would implicitly grow by a factor of `k`.
- `.backward()` is called outside the `autocast` context. This is correct PyTorch practice: `autocast` is needed only for the forward pass; the backward pass automatically uses the same types.
- `tokens_seen += ntok` — the token counter, from which `tokens_to_target` is later derived.

```

# Curvature-based optimizers (Sophia) refresh their diagonal hessian
# every k steps from the (unclipped) gradients already computed.
if hasattr(optimizer, "update_hessian") and step % hessian_update_interval == 0:
    optimizer.update_hessian()
if self.grad_clip:
    torch.nn.utils.clip_grad_norm_(model.parameters(), self.grad_clip)
optimizer.step()
if scheduler is not None:
    scheduler.step()
if device.type == "cuda":
    torch.cuda.synchronize()
step_times.append(time.perf_counter() - t0)

```

There are three important details here.

First: the order "Hessian first, clipping second." The curvature estimate is built from the unclipped gradients. This is deliberate and commented: clipping is a property of the training procedure, not a property of the loss function, and the curvature estimate should reflect the latter. If `update_hessian` came after `clip_grad_norm`, the estimate would be systematically underestimated on steps where clipping fires.

Second: duck typing instead of `isinstance`. `hasattr(optimizer, "update_hessian")` — the trainer knows nothing about the `SophiaG` class, it knows about a protocol. Add

another second-order method tomorrow with the same method, and the trainer will support it without changes.

Third: `torch.cuda.synchronize()` **before timing.** CUDA calls are asynchronous: `optimizer.step()` returns control before the GPU has actually finished the work. Without synchronization, `mean_step_time_s` would measure the speed of queuing tasks rather than the speed of computing them. This is a classic benchmarking mistake, and here it is closed off.

The evaluation block:

```
if step % budget.eval_every_steps == 0 or step == budget.max_steps:
    metrics = self.evaluate()
    metrics.update({"step": step, "tokens": tokens_seen,
                  "train_loss": last_loss})
    history.append(metrics)
    val = metrics.get(eval_metric, metrics["val_loss"])
    if val < best_val:
        best_val, best_step = val, step
    if (target_value is not None and steps_to_target is None
        and val <= target_value):
        steps_to_target, tokens_to_target = step, tokens_seen
    if logger is not None:
        logger.log(metrics)
    model.train()
```

- The condition `or step == budget.max_steps` guarantees an evaluation on the last step, even if it is not a multiple of the period.
- `metrics.get(eval_metric, metrics["val_loss"])` — if the requested metric is absent, we fall back to `val_loss`. This makes the code tolerant of configs where `eval_metric` points at something this task does not compute.
- `steps_to_target is None` in the condition: the **first** crossing of the threshold is recorded, not the last and not the best. That is precisely the definition of "time to target."
- `model.train()` at the end: `evaluate` puts the model into eval mode, and it must be put back (otherwise dropout would stay off for the rest of training).

The finale:

```
peak = (int(torch.cuda.max_memory_allocated(device))
        if device.type == "cuda" else 0)
return RunResult(
    optimizer=opt_name, seed=seed, hparams=hparams or {},
    history=history, best_val=best_val, best_step=best_step,
    steps_to_target=steps_to_target, tokens_to_target=tokens_to_target,
    peak_gpu_mem_bytes=peak,
    mean_step_time_s=sum(step_times) / max(len(step_times), 1),
    final_train_loss=last_loss,
    optimizer_state_bytes=optimizer_state_bytes(optimizer))
```

On CPU the peak memory = 0 (it cannot be measured as simply) — in the `rq1_toy` table you will see `peak_mem_mb = 0.0`, and that is expected.

`max(len(step_times), 1)` is a guard against division by zero when `max_steps=0`.

What makes this file fair

Let us collect in one place everything the trainer guarantees to be identical across the four optimizers: the number of steps, the order and content of the batches (via the seed and the loader's generator), the compute precision, the clipping value, the shape of the LR schedule, the moments and manner of evaluation, and the method of timing. The only things that differ are the `optimizer` object and the `lr` number. That is exactly what needs to be proved in the methodology chapter.

src/optbench/eval/metrics.py

Role. The single place where the loss is computed — shared by training and by landscape analysis.

```
def compute_loss(model, batch, task, model_kind):
    if model_kind == "hf_causal":
        out = model(**batch)
        ntok = int((batch["labels"] != -100).sum().item())
        return out.loss, ntok
    if task == "regression":
        pred = model(batch["x"])
        return F.mse_loss(pred, batch["y"]), batch["y"].numel()
    if task == "causal_lm":
        logits = model(batch["input_ids"])
        vocab = logits.size(-1)
        labels = batch["labels"]
        loss = F.cross_entropy(logits.reshape(-1, vocab),
                               labels.reshape(-1), ignore_index=-100)
        return loss, int((labels != -100).sum().item())
    raise ValueError(...)
```

Three branches for three "model-calling protocols":

1. **HuggingFace:** the model takes an unpacked dictionary `model(**batch)` and computes the loss itself, returning an object with a `.loss` field.
2. **Regression:** the model takes a tensor, and the loss is MSE.
3. **Toy LM:** the model returns logits of shape (B, T, V); they have to be reshaped to (B·T, V) and the labels to (B·T,), and cross-entropy computed.

`ignore_index=-100` is the PyTorch/HuggingFace convention: positions labelled `-100` do not participate in the loss. There are none here (there is no padding), but the code is ready for them.

Returning the pair `(loss, ntok)` is not decoration. The token count is needed in three places: for weighted averaging during evaluation, for perplexity, and for the token-budget counter.

```
@torch.no_grad()
def evaluate(model, loader, task, model_kind, device, autocast=None) -> dict:
    was_training = model.training
    model.eval()
    total, ntok_total = 0.0, 0
    for batch in loader:
        batch = {k: v.to(device) for k, v in batch.items()}
        ctx = autocast() if autocast is not None else _nullctx()
        with ctx:
            loss, ntok = compute_loss(model, batch, task, model_kind)
            total += float(loss.item()) * ntok
            ntok_total += ntok
    mean = total / max(ntok_total, 1)
    out = {"val_loss": mean}
    if task == "causal_lm" or model_kind == "hf_causal":
        out["perplexity"] = math.exp(min(mean, 20.0))
    if was_training:
        model.train()
    return out
```

A walkthrough:

- `@torch.no_grad()` — do not build a computation graph; saves memory and time.
- `model.eval()` turns dropout off and puts normalizations into inference mode.
- **Token weighting:** `total += loss * ntok`, then division by the total number of tokens. A plain average over batches would be biased if the batches were of different sizes. Here, thanks to packing and `drop_last`, all batches are identical, so the result is the same — but the code is correct in the general case too.
- `math.exp(min(mean, 20.0))` — perplexity with overflow protection. $\exp(20) \approx 4.9 \cdot 10^8$; without the cap, a diverging run would give `inf`, which would break averaging and the plots.
- Saving and restoring `model.training` is a courtesy of a library function: it does not leave the model in a changed state.

```
class _nullctx:
    def __enter__(self): return None
    def __exit__(self, *exc): return False
```

A home-made "empty" context manager instead of `contextlib.nullcontext` — probably to avoid pulling in the import. Functionally the same. (In `trainer.py`, incidentally, `contextlib.nullcontext` is used — a small stylistic inconsistency.)

A subtlety about token counting in the HF branch. `ntok` counts all `seq_len` positions, whereas HuggingFace averages the loss over `seq_len - 1` shifted positions. That is, each batch's weight is inflated by the constant factor `seq_len / (seq_len - 1)`. Since the factor is the same for all batches, it cancels in the weighted average and `val_loss` comes out

correct. It is useful to know about this — it is exactly the kind of question examiners like to ask.

src/optbench/run_one.py

Role. "One training run from start to finish." The smallest unit of work, reused by both the sweep and the multi-seed run.

```
def train_one(workload, budget, opt_spec, hparams, runtime, *,
              seed, target_value=None, eval_metric="val_loss"):
    set_global_seed(seed, deterministic=runtime.deterministic)
    model, data = build_experiment_unit(workload, seed=seed)
    trainer = Trainer(model, data, device=runtime.device,
                      precision=runtime.precision, grad_clip=budget.grad_clip)
    hp = dict(hparams)
    lr = hp.pop("lr")
    optimizer = make_optimizer(opt_spec.name, _trainable(model), lr,
                              **{**opt_spec.defaults, **hp})
    scheduler = build_scheduler(optimizer, budget)
    result = trainer.fit(optimizer, budget, target_value=target_value,
                        eval_metric=eval_metric, scheduler=scheduler,
                        opt_name=opt_spec.name, seed=seed,
                        hparams=dict(hparams))
    del model, data, trainer, optimizer, scheduler
    if torch.cuda.is_available():
        torch.cuda.empty_cache()
    return result
```

The order of operations is critical and not accidental:

1. **The seed first, everything else after.** `set_global_seed` is called before the model is created, because weight initialization draws numbers from PyTorch's global generator. Swap these two lines and different optimizers will start from different points, and the whole experiment loses its meaning.
2. `hp = dict(hparams) + hp.pop("lr")` — a copy, so as not to spoil the caller's dictionary. `lr` is pulled out because in `make_optimizer` it is a positional argument.
3. `{**opt_spec.defaults, **hp}` — a dictionary merge: first the optimizer's fixed defaults, then on top of them whatever came from the search space. The order means "the tunable overrides the fixed."
4. `_trainable(model)` — only the parameters with `requires_grad=True` (with LoRA that is 0.3 %).
5. `hparams=dict(hparams)` in `fit` — the original dictionary, `lr` included, is placed into the result, so that the "quality versus LR" curve can be built later.

The final three lines are about the engineering of long runs. Over 72 trials in a row, unreleased GPU memory would accumulate and the last trials would crash with out-of-memory. `del` removes the references, and `empty_cache()` returns cached blocks to the driver.

This is the most important function for understanding fairness. All 92 runs of the main experiment go through exactly this function, differing only in two arguments: `opt_spec` and `hparams["lr"]` (plus `seed` in the multi-seed part). Everything else is taken from the very same `workload`, `budget`, `runtime` objects.

Layer 8. Tuning (RQ1/RQ2)

`src/optbench/tuning/search_space.py`

Role. Turn a declared search space into a concrete list of configurations.

```
def sample_trials(space: SearchSpace, seed: int = 0) -> list[dict]:
    lr = _lr_range(space)
    n = max(1, space.n_trials)
    fixed = {r.name: r.value for r in space.ranges if r.kind == "fixed"}
    if lr.kind in ("log_uniform", "log_grid"):
        if n == 1:
            lrs = [lr.low]
        else:
            lrs = np.logspace(math.log10(lr.low), math.log10(lr.high), n)
    elif lr.kind == "uniform":
        lrs = np.linspace(lr.low, lr.high, n)
    else:
        raise ValueError(f"unsupported lr kind for grid: {lr.kind}")
    return [{"lr": float(v), **fixed} for v in lrs]
```

Despite the name `sample_trials`, there is no random sampling at all — a deterministic logarithmic grid is built. The docstring explains why:

For the LR-only sweep we use a deterministic log-spaced grid over the declared [low, high] range. Even coverage of the LR axis gives both a clean tunability curve (RQ2) and an equal tuning budget per optimizer (RQ1).

The reasoning:

- **For RQ2 a grid is better than random search.** The "quality versus LR" curve must be smooth and cover the axis evenly; random points would give clusters and gaps.
- **For RQ1 a grid is fairer.** With random search, one optimizer might simply "get lucky" and land closer to the optimum.
- **Reproducibility.** A grid does not depend on the random number generator at all.

The `seed` parameter is accepted but not used — which the docstring honestly says ("accepted for API symmetry"). This was done so that switching to random search would not require changing the signature.

Why logarithmic and not linear: the learning rate is a quantity of multiplicative nature. The difference between 1e-4 and 2e-4 is enormous; between 0.5001 and 0.5002 it is imperceptible. `np.logspace` gives equal ratios between neighbouring points: for 18 points over three decades the step is $10^{(3/17)} \approx 1.50$, i.e. each next point is one and a half times larger than the previous one.

`fixed` is a mechanism for adding a constant hyperparameter besides LR to every trial. It is not used in this work's configs, but the schema allows it.

src/optbench/tuning/sweep_runner.py

Role. Run the learning-rate sweep for a single optimizer.

```
def run_equal_budget_sweep(workload, budget, opt_spec, space, runtime, *,
                           sweep_seed=0, target_value=None,
                           eval_metric="val_loss", store=None,
                           run_group="rq1", log=None):
    trials = sample_trials(space, seed=sweep_seed)
    sweep_dir = store.run_dir(run_group, opt_spec.name, "sweep") if store else None
    results = []
    for i, hp in enumerate(trials):
        path = (sweep_dir / f"trial{i:02d}.json") if sweep_dir else None
        if path is not None and path.exists():
            res = result_from_dict(load_json(path))
            results.append(res)
            if log:
                log(f"[{opt_spec.name}] trial {i+1}/{len(trials)} "
                   f"lr={hp['lr']:.2e} (cached) best_val={res.best_val:.4f}")
            continue
        res = train_one(workload, budget, opt_spec, hp, runtime,
                        seed=sweep_seed, target_value=target_value,
                        eval_metric=eval_metric)
        results.append(res)
        if path is not None:
            store.save_json(path, result_to_dict(res))
        if log:
            log(...)
    return results
```

The file's docstring states the invariant once more, now at the sweep level:

every trial uses the SAME workload, budget and sweep_seed (so identical init and data order); only the learning rate changes.

Note `seed=sweep_seed` in the call to `train_one`: **the same seed for all 18 trials**. This is a matter of principle. If each trial ran with its own seed, the difference between trials would be a mixture of "the effect of LR" and "the effect of randomness," and the tunability curve would turn into a noisy sawtooth.

The caching mechanism

Three lines — `if path.exists(): load and skip` — turn a multi-hour run into a resumable one. The power went out on trial 60 out of 72? Re-run the same command, and the first 59 are read back from JSON in seconds.

The same thing makes the run **idempotent**: re-running does not change the result. And the same thing makes it possible to recompute all the tables and plots (`write_tables`, `plot_*`) without a single training step — the data is already on disk.

An important practical warning. The cache key is the trial's ordinal number (`trial00.json`, `trial01.json`, ...), not the learning-rate value. Hence the consequence:

If you change `n_trials`, low or high in the YAML and run again without clearing `results/runs/`, the program will load the old results under the new numbers. Trial #5 of the new grid will get the result of trial #5 of the old grid — with a different LR. **The error will be silent.**

The same applies to `run_multiseed`: the files are named `seed0.json`, `seed1.json`, and if the best LR changed (because the sweep changed), the cached seeds will be from the old configuration.

The working rule: if you change the search space, delete the corresponding folder in `results/runs/`. A more reliable alternative would be to include a hash of the hyperparameters in the filename — this is a good answer to the question "what would you improve?"

`run_group=cfg.name` separates the experiments: the results of `rq1_gpt2_lora` and `rq1b_smollm2_lora` live in different folders and do not get mixed up.

src/optbench/tuning/selection.py

```
def select_best_by_val(results, mode: str = "min"):
    if not results:
        raise ValueError("no results to select from")
    return (min if mode == "min" else max)(results, key=lambda r: r.best_val)
```

Selecting the best trial. `best_val` already contains the required metric (the trainer tracks whichever one is named in `eval_metric`), so the selection is simply a `min`.

A methodological subtlety: the selection is made on the validation set at the sweep seed, whereas the measurement of quality happens afterwards, on other seeds. This separation of "selection" and "evaluation" protects against overfitting to the selection procedure: if the best result were taken from all 18×5 runs, that would be maximization over noise.

```
def tunability_table(results, target=None, metric="val_loss") -> list[dict]:
    rows = [{"lr": r.hparams.get("lr"),
             "best_val": r.best_val,
             "steps_to_target": (steps_to_target_from_history(r.history, target, metric)
                                if target is not None else r.steps_to_target)}
            for r in results]
    rows.sort(key=lambda x: (x["lr"] is None, x["lr"]))
    return rows
```

Builds the data for the RQ2 curve: (lr, quality, steps to target).

The sort key `(x["lr"] is None, x["lr"])` is a trick that sends rows without an LR to the end: `False < True`, so everything that has an `lr` comes first, sorted by value. Without this, the comparison `None < 0.001` would crash with a `TypeError`.

The `target` argument: if it is passed, time-to-target is recomputed from the history; if not, the value recorded during training is used. This is the point at which the ability to change the threshold without retraining is implemented.

Layer 9. Experiment orchestration

src/optbench/experiments/multiseed.py

```
def run_multiseed(workload, budget, opt_spec, best_hparams, runtime, seeds, *,
                  store=None, target_value=None, eval_metric="val_loss",
                  run_group="rq1", log=None):
    seed_dir = store.run_dir(run_group, opt_spec.name, "seeds") if store else None
    results = []
    for seed in seeds:
        path = (seed_dir / f"seed{seed}.json") if seed_dir else None
        if path is not None and path.exists():
            results.append(result_from_dict(load_json(path)))
            continue
        res = train_one(workload, budget, opt_spec, best_hparams, runtime,
                        seed=seed, target_value=target_value,
                        eval_metric=eval_metric)
        results.append(res)
        if path is not None:
            store.save_json(path, result_to_dict(res))
    return results
```

Structurally it is a twin of `sweep_runner`, but the roles of the variables are mirrored:

	What is fixed	What varies
<code>run_equal_budget_sweep</code>	seed	learning rate
<code>run_multiseed</code>	learning rate (the best one)	seed

Why a second run is needed: a single `best_val` number from the sweep is one realization of a random process. To claim "AdamW is better than Lion" you need to know the spread. Five seeds give a sample from which the mean, standard deviation and confidence interval are computed.

Changing the seed changes three things at once: the initialization of the LoRA matrices, the data shuffling order, and dropout. That is exactly the "training randomness" from which we want to separate the effect of the optimizer.

src/optbench/experiments/rq1_benchmark.py

The whole file is one 19-line function, and it is the "plot" of the entire experiment:

```
def run_rq1(cfg, store, log=print):
    out = {}
    for opt_spec in cfg.optimizers:
        space = cfg.search_spaces[opt_spec.name]
        sweep = run_equal_budget_sweep(
            cfg.workload, cfg.budget, opt_spec, space, cfg.runtime,
            sweep_seed=cfg.seeds.sweep_seed,
            target_value=cfg.target_metric_value,
            eval_metric=cfg.workload.eval_metric,
            store=store, run_group=cfg.name, log=log)
        best = select_best_by_val(sweep)
        if log:
            log(f"[{opt_spec.name}] BEST lr={best.hparams.get('lr'):.2e} "
                f"best_val={best.best_val:.4f}")
        seeds = run_multiseed(
            cfg.workload, cfg.budget, opt_spec, best.hparams, cfg.runtime,
            cfg.seeds.final_seeds, store=store,
            target_value=cfg.target_metric_value,
            eval_metric=cfg.workload.eval_metric,
            run_group=cfg.name, log=log)
        out[opt_spec.name] = {"sweep": sweep, "best": best, "seeds": seeds}
    return out
```

Read it as **a definition of the protocol**: for each optimizer — an equal-budget sweep, selection of the best, a re-run over K seeds. Note that `cfg.workload`, `cfg.budget` and `cfg.runtime` are passed into both functions unchanged — the same object, the same reference. No optimizer can get its own version of the task.

The function prints nothing itself: logging arrives as the parameter `log=print`. That is why it can be called from a test with `log=None` and stay silent.

The returned `out` dictionary is the contract between the computational and the analytical parts of the project.

Layer 10. Analysis

src/optbench/analysis/aggregate.py

Role. All of the project's statistics: confidence intervals, the significance test and the recomputation of time-to-target.

The docstring sets the tone:

K is small (5), so we lean on confidence intervals and report the Mann-Whitney p-value with an explicit caveat rather than over-claiming significance.

The bootstrap confidence interval

```
def bootstrap_ci(values, n_boot=5000, alpha=0.05, seed=0):
    v = np.asarray(values, dtype=float)
    if v.size == 0: return (float("nan"), float("nan"))
    if v.size == 1: return (float(v[0]), float(v[0]))
    rng = np.random.default_rng(seed)
    idx = rng.integers(0, v.size, size=(n_boot, v.size))
    means = v[idx].mean(axis=1)
    lo, hi = np.percentile(means, [100 * alpha / 2, 100 * (1 - alpha / 2)])
    return float(lo), float(hi)
```

How the bootstrap works. We have 5 numbers (one per seed). We want to estimate how reliable their mean is. The idea: **5000 times draw 5 values with replacement from those same five**, compute the mean each time, and look at the spread of the resulting 5000 means. The 2.5th and 97.5th percentile bounds give the 95 % interval.

The implementation is vectorized: `rng.integers(0, 5, size=(5000, 5))` builds the whole index matrix at once, `v[idx]` is a 5000×5 matrix of values, `.mean(axis=1)` gives 5000 means. No Python loops, all in one numpy call.

`seed=0` by default — the intervals are reproducible from run to run.

Why the bootstrap and not an ordinary t-interval: a t-interval assumes normality, which is impossible to check at $n=5$. The bootstrap is non-parametric. But it has its own limitation, which is important to understand: with 5 observations there are only $5^5 = 3125$ distinct bootstrap samples, and the interval cannot go beyond the `[min, max]` of the original data. It shows "the uncertainty of the mean over these five seeds," not "the uncertainty over all possible seeds."

The Mann-Whitney test

```
def mann_whitney(a, b) -> dict:
    a = np.asarray(a, dtype=float); b = np.asarray(b, dtype=float)
    if a.size == 0 or b.size == 0:
        return {"U": float("nan"), "p": float("nan")}
    try:
        u, p = stats.mannwhitneyu(a, b, alternative="two-sided")
        return {"U": float(u), "p": float(p)}
    except ValueError:
        return {"U": float("nan"), "p": float("nan")}
```

A non-parametric test: it checks whether one distribution is shifted relative to another, without assuming normality. It works with ranks.

A critically important limitation that you must be able to explain. With small samples the minimum attainable p-value is bounded from below by combinatorics alone:

Sample sizes	Minimum possible p (two-sided)
3 vs 3	0.100
4 vs 4	0.029
5 vs 5	0.0079

This explains both significance tables in the results:

- In `rq1_gpt2_lora_rq1_significance.csv` (5 seeds) the AdamW-Lion and AdamW-Sophia pairs give exactly $p = 0.00794$ — the minimum, i.e. all five AdamW seeds turned out better than all five of the competitor's seeds. The AdamW-SGD pair gives $p = 0.0556$, which corresponds to a single "swap" — on one seed SGD beat AdamW.
- In `rq1b_smollm2_lora_rq1_significance.csv` (3 seeds) all three pairs give $p = 0.1$. This is also the minimum, i.e. perfect separation — but significance at the 0.05 level is in principle unattainable with three seeds.

The correct formulation for the thesis text: "with $K=3$ the Mann-Whitney test cannot give $p < 0.05$ for any outcome, so for the second model we rely on confidence intervals and on the consistency of the ranking rather than on the p-value."

Recomputing time-to-target

```
def steps_to_target_from_history(history, target, metric="val_loss"):
    """First eval step where `metric <= target`, derived from a run's eval history.
    Recomputing time-to-target from the stored history (rather than the value baked
    into RunResult at fit time) lets the target be recalibrated without re-running.
    Returns None if the target is never reached (honest "did not reach")."""
    if target is None or not history:
        return None
    for h in history:
        val = h.get(metric)
        if val is not None and val <= target:
            return h.get("step")
    return None
```

A small function with great methodological significance. The threshold `target_metric_value` is a quantity that cannot be chosen correctly before the experiment: until you see where the optimizers converge, you cannot tell which level is meaningful. But revising the threshold after the experiment is dangerous — it looks like fitting.

The project's solution: **store the full evaluation history of every run and recompute the metric from it**. Then changing the threshold means changing one line in the YAML and re-running the analytics in a second, without a single training step. And, more importantly, the rule for choosing the threshold is stated explicitly ("a small margin above the level of the weakest participant") rather than tuned to the desired answer.

Returning `None` instead of, say, `max_steps` is honesty: "did not reach" and "reached at the last step" are different events, and substituting the second for the first would distort the mean.

General aggregation

```
def aggregate_metric(results, getter) -> dict:
    vals = np.asarray([getter(r) for r in results], dtype=float)
    lo, hi = bootstrap_ci(vals)
    return {"mean": float(vals.mean()) if vals.size else float("nan"),
            "std": float(vals.std(ddof=1)) if vals.size > 1 else 0.0,
            "n": int(vals.size), "ci_low": lo, "ci_high": hi}
```

A generic "compute statistics over a list of results" function, where `getter` is a function that extracts the required field. `ddof=1` gives the sample standard deviation (dividing by $n-1$, not n). This is the right choice: five seeds are a sample, not the whole population.

Curiously, `write_tables` does not use this function but computes the same thing by hand. A small duplication that would be worth eliminating.

src/optbench/analysis/tables.py

Role. Turn the `out` dictionary into three CSV tables for the "Results" chapter.

```
def write_tables(out: dict, tables_dir: str, cfg) -> dict:
    os.makedirs(tables_dir, exist_ok=True)
    main_rows, tun_rows = [], []
    seed_best = {}
    target = cfg.target_metric_value
    metric = cfg.workload.eval_metric
    for opt, d in out.items():
        seeds, best = d["seeds"], d["best"]
        bvals = [r.best_val for r in seeds]
        seed_best[opt] = bvals
        lo, hi = bootstrap_ci(bvals)
        n_reached = sum(steps_to_target_from_history(r.history, target, metric)
                        is not None for r in seeds)
        main_rows.append({
            "optimizer": opt,
            "best_lr": best.hparams.get("lr"),
            "val_mean": float(np.mean(bvals)) if bvals else None,
            "val_std": float(np.std(bvals, ddof=1)) if len(bvals) > 1 else 0.0,
            "val_ci_low": lo, "val_ci_high": hi,
            "steps_to_target_mean": _mean([...]),
            "seeds_reaching_target": f"{n_reached}/{len(seeds)}",
            "peak_mem_mb": _mean([r.peak_gpu_mem_bytes / 1e6 for r in seeds]),
            "step_time_ms": _mean([r.mean_step_time_s * 1e3 for r in seeds]),
            "opt_state_mb": _mean([r.optimizer_state_bytes / 1e6 for r in seeds]),
            "n_seeds": len(bvals)})
        for row in tunability_table(d["sweep"], target=target, metric=metric):
            tun_rows.append({"optimizer": opt, **row})
```

The `seeds_reaching_target` column is an important fairness detail. The mean time-to-target is computed only over those seeds that actually reached the target (the `_mean` function discards `Nones`). Without the second column a reader could not tell "500 steps, all five seeds" from "500 steps, but only one got there." In the results of the main experiment it is 5/5 everywhere — i.e. the threshold of 3.20 was chosen well.

```
def _mean(xs):
    xs = [x for x in xs if x is not None]
    return float(np.mean(xs)) if xs else None
```

A helper "mean, ignoring missing values." It returns `None` rather than `nan` if all values are missing — which in the CSV produces an empty cell.

```
ref = "adamw" if "adamw" in seed_best else next(iter(seed_best))
sig_rows = []
for opt, vals in seed_best.items():
    if opt == ref: continue
    sig_rows.append({"pair": f"{ref}_vs_{opt}",
                    **mann_whitney(seed_best[ref], vals)})
```

All comparisons are made **against AdamW as the reference**, not "everyone against everyone." This is a deliberate choice: AdamW is the de facto standard, and the work's question is framed as "does anyone beat the standard," not "how do they all relate to each other." A side benefit: three comparisons instead of six, i.e. a smaller multiple-

comparisons problem. `next(iter(...))` is a fallback in case AdamW is not in the experiment.

Three output files: `{name}_rq1_main.csv`, `{name}_rq2_tunability.csv`, `{name}_rq1_significance.csv`. The prefix is the experiment name, so tables from different experiments do not overwrite each other.

src/optbench/analysis/plots/init.py

```
def ensure_parent(path: str) -> str:
    os.makedirs(os.path.dirname(os.path.abspath(path)), exist_ok=True)
    return path
```

Four lines: create the folder for the file and return the path. It lets you write `fig.savefig(ensure_parent(path))` in one line in all three plotting modules.

src/optbench/analysis/plots/curves.py

```
matplotlib.use("Agg")

def plot_val_curves(out: dict, save_path: str) -> str:
    fig, ax = plt.subplots(figsize=(7, 4.5))
    for opt, d in out.items():
        seeds = d["seeds"]
        if not seeds: continue
        length = min(len(r.history) for r in seeds)
        if length == 0: continue
        steps = [seeds[0].history[i]["step"] for i in range(length)]
        mat = np.array([[r.history[i]["val_loss"] for i in range(length)]
                        for r in seeds])
        mean, std = mat.mean(axis=0), mat.std(axis=0)
        ax.plot(steps, mean, label=opt)
        ax.fill_between(steps, mean - std, mean + std, alpha=0.2)
    ...
```

`matplotlib.use("Agg")` before importing pyplot switches to a backend without a graphical window. This is mandatory when running on a server with no display; without it matplotlib would try to open a window and crash.

`length = min(len(r.history) for r in seeds)` truncates to the shortest history. A safeguard in case one run was interrupted early: otherwise `np.array` built from rows of different lengths would give an object array and everything downstream would break.

`fill_between(mean - std, mean + std, alpha=0.2)` is the semi-transparent spread band. It is exactly what visually answers the question "is the difference between optimizers larger than the noise or not": if the bands overlap, the difference is not convincing.

src/optbench/analysis/plots/tunability.py

The main plot for RQ2, and it is made with an understanding of the scale problem:

```

def plot_tunability(out: dict, save_path: str, zoom_span: float = 0.20) -> str:
    """Two panels: full range (left) shows divergence at bad LRs; the zoomed-Y panel
    (right) reveals the basin shape near the optima, which the full scale compresses
    into an unreadable band when one optimizer blows up."""
    series = {}
    for opt, d in out.items():
        rows = tunability_table(d["sweep"])
        xs = [r["lr"] for r in rows if r["lr"] is not None]
        ys = [r["best_val"] for r in rows if r["lr"] is not None]
        series[opt] = (xs, ys)
    ymin = min(min(ys) for _, ys in series.values() if ys)
    fig, (ax_full, ax_zoom) = plt.subplots(1, 2, figsize=(12, 4.5))
    for opt, (xs, ys) in series.items():
        ax_full.plot(xs, ys, marker="o", markersize=4, label=opt)
        ax_zoom.plot(xs, ys, marker="o", markersize=4, label=opt)
    for ax in (ax_full, ax_zoom):
        ax.set_xscale("log")
        ...
    ax_zoom.set_ylim(ymin - 0.005, ymin + zoom_span)

```

The problem the two panels solve: at a bad LR Sophia diverges and its loss flies up to 15.5. If everything is drawn at one scale, all the interesting differences (between 3.176 and 3.188) are squeezed into a single indistinguishable line. On the left is the full range, where the divergence is visible; on the right is the same curve with a clipped Y axis, where the profile of the "basin" near the optimum is visible.

This is exactly the content of RQ2: **the width of the basin**. From the right-hand panel you can see that AdamW's floor is wide (quality barely changes across a decade of LR), Lion's is narrower, Sophia's has a sharp cliff on the right, and SGD's "basin" is in an entirely different LR region.

Note that `tunability_table(d["sweep"])` is called without `target` — the plot needs only `lr` and `best_val`, and the time-to-target column is not used.

src/optbench/analysis/plots/landscape.py

Three functions for RQ3.

```

def plot_surface_2d(coords, Z, save_path, title="Loss surface"):
    X, Y = np.meshgrid(coords, coords, indexing="ij")
    fig, ax = plt.subplots(figsize=(5.5, 4.5))
    cf = ax.contourf(X, Y, Z, levels=30, cmap="viridis")
    ax.contour(X, Y, Z, levels=12, colors="k", linewidths=0.3, alpha=0.5)
    ax.plot(0, 0, "r*", markersize=12, label="solution")
    fig.colorbar(cf, ax=ax, label="loss")
    ...

```

`indexing="ij"` is matrix indexing instead of Cartesian ("`xy`", the default). It matters that `Z[i, j]` corresponds to `coords[i]` along the first axis and `coords[j]` along the second, which is how it is filled in `loss_surface_2d`. A mistake here would give a transposed picture.

The `contourf` fill plus thin black `contour` isolines is a standard technique: colour gives the general impression, lines give the exact structure. The red star at (0,0) is the solution itself.

```
def plot_compare_surfaces(panels, save_path, suptitle=""):
    """panels: list of (title, coords, Z). Side-by-side contours,
    each on its own scale."""
```

Each panel has its own colour scale — and this is a deliberate decision you must be able to talk about. On the one hand, a shared scale would allow absolute values to be compared. On the other hand, what is being compared here is the **shape** (how steeply the loss rises as you move away from the solution), not the level; with a shared scale the panel with the larger range would "eat" the whole palette and the rest would become monochrome. On the plot `rq3_naive_vs_filtornorm.png` this is visible: on the left the scale goes up to 5.4, on the right up to 1.9 — and it is precisely this contrast of ranges that is the result.

```
def plot_interpolation(ts, series, save_path, title=...):
    for label, ys in series.items():
        ax.plot(ts, ys, marker="o", markersize=3, label=label)
    ax.axvline(0.0, color="gray", ls="--", lw=0.8)
    ax.axvline(1.0, color="gray", ls="--", lw=0.8)
    ax.set_xlabel("interpolation coefficient t (0 = solution A, 1 = solution B)")
```

The vertical dashed lines mark the positions of the two solutions. Everything between them is the "road" from one to the other; a bump on that road means a barrier, i.e. the solutions lie in different basins and are not connected by a straight line of low loss.

Layer 11. Loss landscape (RQ3)

src/optbench/landscape/params.py

```
def get_params(model) -> list[torch.Tensor]:
    """Detached clone of every parameter, in module order (a landscape point)."""
    return [p.detach().clone() for p in model.parameters()]

@torch.no_grad()
def set_params(model, point) -> None:
    for p, v in zip(model.parameters(), point):
        p.copy_(v)

@torch.no_grad()
def set_offset(model, base, dirs, coeffs) -> None:
    """Set params to base + sum_k coeffs[k] * dirs[k] (in place)."""
    for i, p in enumerate(model.parameters()):
        val = base[i].clone()
        for c, d in zip(coeffs, dirs):
            if c != 0.0:
                val = val + c * d[i]
        p.copy_(val)
```

The representation chosen in the project: both a "point" θ and a "direction" d are lists of tensors aligned with the order of `model.parameters()`. The alternative — flattening everything into one long vector — would have simplified the arithmetic but would have required constantly remembering the shapes for reassembly. The docstring explains the choice: "keeps the slicing/interpolation code shape-safe without ever flattening to one giant vector."

`detach().clone()` in `get_params`: `detach` unhooks from the computation graph, `clone` makes a genuine copy. Without `clone` we would get a reference to the live weights, and after the next training step the "saved solution" would change by itself.

The `_` suffix in `set_params_` and `set_offset_` follows the PyTorch convention: the function modifies its argument in place.

The `if c != 0.0` optimization in `set_offset_` skips zero coefficients. On a 25×25 grid a coordinate is often zero (the entire central cross), so the saving is noticeable.

src/optbench/landscape/directions.py

Role. This is where the scientific content of RQ3 lives. The docstring states it directly:

A raw random direction has a scale unrelated to the local weight magnitudes, so a slice along it says nothing comparable about flatness. Filter normalization rescales the direction per parameter tensor to match that tensor's norm, which is what makes flatness comparisons between solutions meaningful.

The problem

We want to draw "what the loss function looks like around the solution we found" and compare whose minimum is sharper. We take a random direction d , move along it by α and look at $L(\theta + \alpha d)$. The question: what does $\alpha = 0.5$ mean?

If the weights of one solution have norm 100 and those of another have norm 1, then a step of the same absolute distance means a completely different relative perturbation. The first solution will look "flat" simply because its weights are larger. The comparison is meaningless.

The solution: filter normalization

```
def filter_normalize(direction, params, ignore_dims_lt: int = 2):
    out = []
    for d, p in zip(direction, params):
        if p.dim() < ignore_dims_lt:
            out.append(torch.zeros_like(d))
            continue
        pn, dn = p.norm(), d.norm()
        if dn.item() == 0.0 or pn.item() == 0.0:
            out.append(torch.zeros_like(d))
        else:
            out.append(d * (pn / dn))
    return out
```

For each parameter tensor separately, the direction is rescaled so that $\|d_i\| = \|\theta_i\|$. Then $\alpha = 0.1$ means "shift every weight tensor by 10 % of its own norm" — a quantity that is comparable across any solutions and any models.

The `if p.dim() < ignore_dims_lt` branch zeroes the direction for one-dimensional parameters: biases and normalization coefficients. This is Li et al.'s recommendation: such parameters have a small norm but a large sensitivity, and without zeroing they would dominate the slice.

A comment clarifies the adaptation of the method: in the original paper the normalization was done per filter for convolutional layers (each filter separately); for fully connected weight matrices the generally accepted adaptation is to take the Frobenius norm of the whole matrix, i.e. to treat the matrix as one group. That is exactly what is done here, and it is stated explicitly.

The naive alternative (the baseline)

```
def naive_normalize(direction, params):
    total_p = torch.sqrt(sum((p.float() ** 2).sum() for p in params))
    total_d = torch.sqrt(sum((d.float() ** 2).sum() for d in direction))
    scale = 0.0 if total_d.item() == 0.0 else (total_p / total_d).item()
    return [d * scale for d in direction]
```

Here there is a single global scaling factor for everything: the total norm of the direction is set equal to the total norm of the parameters. The total step size comes out the same as with filter normalization — and that is the crux of the experiment.

What breaks as a result: a random Gaussian vector distributes its "energy" in proportion to a tensor's number of elements, not its norm. Large tensors (embeddings) receive almost all of the perturbation, small ones almost none. The perturbation stops corresponding to the structure of the model.

The RQ3 experiment in `run_landscape.py` is set up as a **controlled comparison**: the same solution, the same random direction, the same total step size — only the normalization method differs. And the pictures come out noticeably different (a loss range of 5.4 versus 1.9). The conclusion: the "sharpness" of the minimum in such a picture is an artifact of the normalization, not a property of the solution.

`src/optbench/landscape/surface.py`

```
def make_eval_loss(model, loader, task, model_kind, device="cpu", autocast=None):
    """A zero-arg closure returning mean loss over loader at current params."""
    def _f() -> float:
        return float(evaluate(model, loader, task, model_kind,
                               device, autocast)["val_loss"])
    return _f
```

A closure: what is returned is a function with no arguments that computes the loss at the model's current weights. Downstream, the grid-scanning code knows nothing about either the model or the data — it only moves the weights and calls `eval_loss()`. A clean separation of responsibilities.

```
@torch.no_grad()
def loss_surface_2d(model, eval_loss, theta, dir_x, dir_y, coords):
    coords = np.asarray(coords, dtype=float)
    Z = np.empty((len(coords), len(coords)), dtype=float)
    for i, a in enumerate(coords):
        for j, b in enumerate(coords):
            set_offset_(model, theta, [dir_x, dir_y], [float(a), float(b)])
            Z[i, j] = eval_loss()
    set_params_(model, theta) # restore
    return coords, Z
```

A double loop over the grid: at each point the weights are set to $\theta + a \cdot dx + b \cdot dy$ and the loss is computed. With `coords` of 25 points this is 625 full passes over the validation set — hence the decision to do RQ3 only on the toy model.

`set_params_(model, theta)` at the end is the restoration. This is not a trifle: without it the function would leave the model at the last grid point, and the next call would work from a spoiled solution. The tests in `test_landscape.py` specifically check the restoration.

```

@torch.no_grad()
def loss_interpolation_1d(model, eval_loss, theta_a, theta_b, ts):
    delta = [b - a for a, b in zip(theta_a, theta_b)]
    ...
    for k, t in enumerate(ts):
        set_offset_(model, theta_a, [delta], [float(t)])
        ys[k] = eval_loss()
    set_params_(model, theta_a)
    return ts, ys

```

Linear interpolation $(1-t)\cdot\theta_A + t\cdot\theta_B$, rewritten as $\theta_A + t\cdot(\theta_B - \theta_A)$ — the same thing, but reusing the ready-made `set_offset_` function.

The range of `t` in `run_landscape.py` is from -0.25 to 1.25 , i.e. it goes slightly beyond the segment between the solutions. The extrapolation helps to see whether both ends really do lie in local minima.

The substantive interpretation, recorded in the docstring: "A bump between the endpoints is a loss barrier (the two solutions sit in different basins)." A monotone descent between the solutions would mean they are connected by a low path (one basin); a bump means they are not.

Layer 12. Infrastructure

src/optbench/logging/artifacts.py

```

class ArtifactStore:
    def __init__(self, root):
        self.root = Path(root)
        self.root.mkdir(parents=True, exist_ok=True)
    def run_dir(self, *parts: str) -> Path:
        p = self.root.joinpath(*parts)
        p.mkdir(parents=True, exist_ok=True)
        return p

```

`run_dir("rq1_gpt2_lora", "adamw", "sweep")` creates and returns `results/runs/rq1_gpt2_lora/adamw/sweep/`. The "experiment / optimizer / phase" hierarchy is what makes the cache understandable when browsing it in a file manager.

```

@staticmethod
def save_json(path, obj) -> None:
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    with open(path, "w", encoding="utf-8") as fh:
        json.dump(obj, fh, indent=2, default=str)

```

`indent=2` makes the files human-readable (you can open one and check a trial by eye). `default=str` means "if you do not know how to serialize it, convert it to a string":

insurance against crashing on numpy types and Path. Convenient insurance, but slightly dangerous: it can silently turn a number into a string. In this code it never fires, because `lr` is explicitly cast to `float` in `sample_trials`.

```
@staticmethod
def save_csv(path, rows) -> None:
    if not rows:
        path.write_text("", encoding="utf-8"); return
    fields = list(dict.fromkeys(k for r in rows for k in r))
    with open(path, "w", newline="", encoding="utf-8") as fh:
        w = csv.DictWriter(fh, fieldnames=fields)
        w.writeheader(); w.writerows(rows)
```

`dict.fromkeys(...)` is the "unique values, order preserved" idiom. It walks all the rows, collects all the keys, removes duplicates, but keeps the order of first appearance. That way the column order in the CSV matches the order in which they were put into the dictionary.

`newline=""` in `open` is mandatory for the `csv` module on Windows: otherwise you get doubled line breaks.

The three `@staticmethods` — these functions do not use `self`; they are static simply so as to live next to the store class.

src/optbench/serialize.py

```
def result_to_dict(res: RunResult) -> dict:
    return asdict(res)

def result_from_dict(d: dict) -> RunResult:
    return RunResult(**d)
```

Four lines, but they are what make the cache possible. `dataclasses.asdict` recursively turns a dataclass into a dictionary, and `RunResult(**d)` turns it back.

A fragile spot you need to know about: if you add a new field to `RunResult` without a default value, all the old JSON files will stop loading (`TypeError: missing required argument`). That is exactly why the last field is declared as `optimizer_state_bytes: int = 0` — with a default, so that results saved before it appeared can still be read. This is manual format-versioning.

src/optbench/reproducibility/seeding.py

```
def set_global_seed(seed: int, deterministic: bool = True) -> None:
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)
    if deterministic:
        torch.backends.cudnn.deterministic = True
        torch.backends.cudnn.benchmark = False
        os.environ.setdefault("CUBLAS_WORKSPACE_CONFIG", ":4096:8")
        try:
            torch.use_deterministic_algorithms(True, warn_only=True)
        except Exception:
            pass
```

Four independent random-number generators are fixed one after another: the standard `random`, `numpy`, `torch`'s CPU generator and all GPU generators. Skipping any one of them means losing reproducibility somewhere.

Then come three levels of the fight against CUDA non-determinism:

- `cudnn.deterministic = True` — use only deterministic convolution algorithms.
- `cudnn.benchmark = False` — do not pick the algorithm automatically by benchmarking (auto-selection can choose different algorithms on different runs).
- `CUBLAS_WORKSPACE_CONFIG=":4096:8"` — the environment variable NVIDIA requires for deterministic matrix multiplications. `setdefault` means "do not overwrite it if the user has set their own."
- `use_deterministic_algorithms(True, warn_only=True)` — the global switch; `warn_only` means "if some operation has no deterministic version, warn but do not crash."

The docstring honestly admits the limits: full determinism on a GPU is unattainable, so the project additionally insures itself with multi-seed runs and confidence intervals. This is the correct scientific position: not "everything is reproducible down to the last bit," but "we fix everything we can and measure the residual noise."

The test `test_seeding_determinism.py` checks bit-exact agreement of two runs on CPU, where that is attainable.

src/optbench/reproducibility/env.py

```
def capture_env() -> dict:
    import numpy, torch
    env = {"python": platform.python_version(),
          "platform": platform.platform(),
          "torch": torch.__version__,
          "numpy": numpy.__version__,
          "cuda_available": torch.cuda.is_available()}
    try:
        import transformers
        env["transformers"] = transformers.__version__
    except Exception:
        pass
    if torch.cuda.is_available():
        env["gpu"] = torch.cuda.get_device_name(0)
        env["cuda_capability"] = list(torch.cuda.get_device_capability(0))
        env["cuda"] = torch.version.cuda
    return env
```

A snapshot of the environment: the versions of Python, the OS, torch, numpy, transformers, the GPU model and the CUDA version. It is printed at the start of every run and saved to `env.json` next to the results.

Why this is needed: a year from now, looking at the number "3.1763," you will be able to say on what hardware and with which library versions it was obtained. Different PyTorch versions can give slightly different numbers, and without such a snapshot the discrepancy would be impossible to explain.

The `try/except` around `transformers` — the library may not be installed (the toy path gets by without it), and the environment snapshot should not crash because of that.

Layer 13. Scripts (entry points)

All five scripts are built the same way: parse the arguments or set the constants, call the library, print the result. There is no logic in them — and that is right.

scripts/run_rq1.py — the main script

```
def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--config", required=True)
    ap.add_argument("--tables", default="results/tables")
    ap.add_argument("--figures", default="results/figures")
    args = ap.parse_args()
    cfg = load_experiment(args.config)
    store = ArtifactStore(cfg.runtime.results_root)
    print(f"env: {capture_env()}")
    print(f"experiment: {cfg.name} | optimizers: {[o.name for o in cfg.optimizers]}")
    out = run_rq1(cfg, store, log=print)
    tables = write_tables(out, args.tables, cfg)
    tun = plot_tunability(out, f"{args.figures}/{cfg.name}_rq2_tunability.png")
    curves = plot_val_curves(out, f"{args.figures}/{cfg.name}_rq1_val_curves.png")
    print("\n=== RQ1 main (per optimizer) ===")
    for row in tables["main"]:
        print(f"{row['optimizer']:>13} best_lr={row['best_lr']:.2e} "
              f"val={row['val_mean']:.4f} +/- {row['val_std']:.4f} ...")
```

Nine substantive lines for the whole main experiment of the thesis. The same script serves four different experiments — only `--config` changes. This is a direct consequence of all the variability having been moved into YAML.

`log=print` — this is where the print function is "injected" into the library. The library stays pure and the script gets a detailed log.

The output filenames are built from `cfg.name`, so running a different config does not overwrite previous results.

scripts/run_landscape.py — RQ3

The only script with substantive logic, because RQ3 is not described by the YAML schema. The configuration is set directly in code as module constants:

```
SEED = 0
WORKLOAD = WorkloadConfig(name="toy_char_landscape", task="causal_lm",
    model_kind="toy_transformer", seq_len=48, micro_batch_size=16,
    extras={"d_model": 64, "nhead": 2, "num_layers": 2})
BUDGET = BudgetConfig(max_steps=400, eval_every_steps=200, warmup_ratio=0.05,
    scheduler="cosine", grad_clip=1.0)
RUNTIME = RuntimeConfig(device="cpu", precision="fp32")
OPTS = [("adamw", 3e-3, {...}),
    ("sgd_momentum", 0.2, {...}),
    ("lion", 1e-3, {...})]
```

The comment next to `OPTS` is important:

Same toy LRs are fine here -- we want trained solutions to slice, not a fair benchmark.

That is, no fair comparison is being made here: the goal is to obtain three trained solutions in order to slice them. Tuning the LR would be a waste. These two regimes must be distinguished clearly: RQ1/RQ2 are about the fairness of the comparison, RQ3 is about the fairness of the visualization.

```
def train_solution(opt_name, lr, defaults):
    set_global_seed(SEED)
    model, data = build_experiment_unit(WORKLOAD, seed=SEED) # same init + data
    trainer = Trainer(model, data, device=RUNTIME.device, ...)
    params = [p for p in model.parameters() if p.requires_grad]
    optimizer = make_optimizer(opt_name, params, lr, **defaults)
    scheduler = build_scheduler(optimizer, BUDGET)
    res = trainer.fit(optimizer, BUDGET, scheduler=scheduler, ...)
    return model, data, res.best_val
```

The invariant is nonetheless reused: `set_global_seed(SEED)` before each training run means that all three solutions start from the same point and see the data in the same order. Otherwise the interpolation between them would mix "different optimizers" with "different initializations."

```
def main():
    solutions = {}
    host_model = host_data = None
    for name, lr, defaults in OPTS:
        model, data, best = train_solution(name, lr, defaults)
        solutions[name] = get_params(model)
        if host_model is None:
            host_model, host_data = model, data # evaluate every slice on one host
```

A subtle and important technique. Three models are trained, but only the weights are kept (`get_params` is a copy of the tensors). Afterwards all computations run on a single "host model" (the first one), into which each solution's weights are loaded in turn. This is legitimate because all three models have the same architecture, and it guarantees that all slices are computed on one and the same validation loader — i.e. on the same data.

```
eval_loss = make_eval_loss(host_model, host_data.val_loader,
                           host_data.task, "toy_transformer")
gen = torch.Generator().manual_seed(123)
coords = np.linspace(-0.5, 0.5, 25)
```

A separate generator with a fixed seed for the random directions — the pictures are reproducible. A 25×25 grid over ± 0.5 , where 0.5 after filter normalization means "shift every weight tensor by half its norm."

Figure A — one slice per optimizer, all with filter normalization:

```

panels = []
for name in solutions:
    theta = solutions[name]
    dx = filter_normalize(random_direction(theta, gen), theta)
    dy = filter_normalize(random_direction(theta, gen), theta)
    _, Z = loss_surface_2d(host_model, eval_loss, theta, dx, dy, coords)
    panels.append((name, coords, Z))

```

Note: the directions are different for each solution (the generator keeps producing new numbers). This is correct — with filter normalization what matters is not the particular direction but the fact that it is normalized relative to the given solution; averaging over random directions is the very point of the method.

Figure B — the main RQ3 result:

```

theta = solutions["adamw"]
raw_x = random_direction(theta, gen)
raw_y = random_direction(theta, gen)
fn_x, fn_y = filter_normalize(raw_x, theta), filter_normalize(raw_y, theta)
nv_x, nv_y = naive_normalize(raw_x, theta), naive_normalize(raw_y, theta)
_, Z_fn = loss_surface_2d(host_model, eval_loss, theta, fn_x, fn_y, coords)
_, Z_nv = loss_surface_2d(host_model, eval_loss, theta, nv_x, nv_y, coords)

```

Note the rigour of the experiment: `raw_x` and `raw_y` are generated **once** and then normalized in two ways. This means the difference between the panels cannot be explained by different random directions — the only difference is the normalization method. This is a controlled experiment in its purest form.

Figure C — interpolation between the AdamW and SGD solutions over t from -0.25 to 1.25 .

```

set_params_(host_model, solutions["adamw"]) # leave host clean

```

The final line returns the host model to a meaningful state. Formally unnecessary (the program terminates right after), but it is a good habit.

A minor remark. `plot_surface_2d` is imported but not used — all three figures use `plot_compare_surfaces` and `plot_interpolation`. A harmless "dead" import.

scripts/baseline_gpt2.py — calibrating the threshold

A single GPT-2 + LoRA run with AdamW. Its purpose is explained in the docstring:

```

Purpose: see where validation loss lands under a realistic-but-cheap budget so we
can pick target_metric_value for the time-to-target metric used in the sweeps.

```

That is, this is a preparatory script: it answers the question "what loss level is attainable at all" before the threshold is chosen.

```
target = round(init["val_loss"] - 0.9 * (init["val_loss"] - res.best_val), 2)
print(f"suggested target_metric_value (val_loss) for sweeps: {target}")
```

The heuristic: take the level corresponding to 90 % of the way from the initial loss to the best one achieved. Note that the final threshold in the config (3.20) was chosen not by this formula but by a different rule — "slightly above the level of the weakest optimizer." The script gave the first estimate (3.36), it turned out to be too easy, and the rule was changed. This whole story is recorded in the comments of `rq1_gpt2_lora.yaml` — a rare and valuable example of the methodological path being documented openly.

The script also saves a full set of artifacts:

```
store.save_json(run_dir / "config.json", asdict(workload))
store.save_json(run_dir / "env.json", capture_env())
store.save_jsonl(run_dir / "history.jsonl", [{"step": 0, **init}] + res.history)
store.save_json(run_dir / "summary.json", {...})
```

Four files: what was run, where it was run, what happened step by step, and what came out. `[{"step": 0, **init}] + res.history` prepends the pre-training evaluation as step 0, so that the curve starts from the model's initial quality.

scripts/smoke_hf.py — a quick check of the GPU path

20 steps, 128 data blocks, `grad_accum_steps=2` (the only place in the project where gradient accumulation is actually used). It checks the entire chain: model loading, the LoRA wrapper, bf16, accumulation, the schedule, evaluation, and memory measurement.

```
assert res.best_val < init["val_loss"], "loss did not decrease"
print("SMOKE OK")
```

It ends with an assertion and prints a marker. This is a "test" that cannot be included in `pytest`, because it requires a GPU and downloading a model — hence it is packaged as a separate script. The line `SMOKE OK` is convenient for automated checking in deployment scripts.

scripts/make_figures_pdf.py — assembling the illustrations

```
FIGURES = [
    (f"{FIG}/rq1_gpt2_lora_rq1_val_curves.png",
     "RQ1 - Validation curves (GPT-2 + LoRA)",
     "Best config per optimizer, mean +-std over 5 seeds. ..."),
    ...
]

def main():
    with PdfPages(OUT) as pdf:
        for path, title, caption in FIGURES:
            if not os.path.exists(path):
                print(f"skip (not found): {path}")
                continue
            img = mpimg.imread(path)
            fig = plt.figure(figsize=(11, 8.5))
            ax = fig.add_axes([0.03, 0.06, 0.94, 0.82])
            ax.imshow(img); ax.axis("off")
            ax.set_title(title, fontsize=14, fontweight="bold", pad=12)
            fig.text(0.5, 0.03, caption, ha="center", fontsize=9, wrap=True)
            pdf.savefig(fig); plt.close(fig)
```

It collects all the PNGs into one PDF — one illustration per page with a title and a caption. Skipping missing files (`continue`) makes it possible to run the assembly at any moment, even while some experiments are still computing.

The `FIGURES` list contains ready-made captions with interpretations — effectively a draft of the figure captions for the thesis. For example: "AdamW flattest, Lion sharpest" or "a loss barrier (~ 0.027) means they sit in different basins."

Layer 14. Tests

Eight test files, all of which run on CPU in seconds. They do not check "the correctness of the science" — they check that the instrument does not lie.

test_config_loader.py

```
def test_load_experiment():
    cfg = load_experiment(CONFIG)
    assert cfg.name == "rq1_toy"
    assert cfg.workload.model_kind == "toy_mlp"
    assert cfg.workload.extras["in_dim"] == 8
    assert [o.name for o in cfg.optimizers] == ["adamw", "lion"]
    assert cfg.search_spaces["adamw"].ranges[0].name == "lr"
    assert cfg.seeds.final_seeds == (0, 1)
```

Checks that the YAML unfolds into the correct nested structure, including the conversion of the seed list into a tuple. A real config from `configs/` is loaded rather than a made-up one — which means the test also checks that the project's config is not broken.

test_search_space.py

```
def test_grid_count_range_and_order():
    trials = sample_trials(_space(n=5, low=1e-4, high=1e-1))
    assert len(trials) == 5
    lrs = [t["lr"] for t in trials]
    assert lrs[0] == 1e-4 and abs(lrs[-1] - 1e-1) < 1e-9
    assert all(lrs[i] < lrs[i + 1] for i in range(len(lrs) - 1))

def test_deterministic():
    assert sample_trials(_space()) == sample_trials(_space())
```

Four properties of the grid: the correct number of points (which is precisely the "equal budget"), inclusion of both endpoints, monotonic increase, and determinism. The last test is a direct check of reproducibility.

test_optimizers.py

```
OPTIMIZERS = [("sgd_momentum", 5e-2, {}), ("adamw", 1e-2, {}),
              ("lion", 3e-3, {}), ("sophia", 1e-2, {"rho": 0.05})]

@pytest.mark.parametrize("name,lr,kwarg", OPTIMIZERS)
def test_optimizer_reduces_loss(name, lr, kwarg):
    ...
    init_val = trainer.evaluate()["val_loss"]
    opt = make_optimizer(name, model.parameters(), lr, **kwarg)
    res = trainer.fit(opt, BudgetConfig(max_steps=200, eval_every_steps=100),
                     opt_name=name)
    assert res.best_val < init_val * 0.7, f"{name} failed to reduce loss"
    assert optimizer_state_bytes(opt) > 0
```

`@pytest.mark.parametrize` generates four separate tests from one body. Each checks two things: that the optimizer really does reduce the loss (by at least 30 %) and that it allocates state.

This is the main test for the in-house Sophia implementation. It does not prove that it is implemented exactly as in the paper, but it catches gross errors: a wrong sign, division instead of multiplication, state that never gets filled. A strict check (comparison against a reference implementation) would be a substantial strengthening of the work.

test_smoke_trainer.py

```
def test_regression_loss_decreases():
    ...
    assert res.best_val < init_val * 0.5

def test_causal_lm_loss_decreases():
    ...
    assert res.best_val < init_val
    assert "perplexity" in res.history[-1]
```

Checks both branches of the task: regression and language modelling. The second test also confirms that perplexity is computed for LM tasks and not computed for regression.

test_seeding_determinism.py

```
def test_determinism():
    a_hist, a_best = _run_once()
    b_hist, b_best = _run_once()
    assert a_hist == b_hist
    assert a_best == b_best
```

Two identically seeded runs must produce bit-identical loss trajectories. The comparison uses `==` on lists of floats — i.e. exact equality is required, not approximate. On CPU this is attainable.

This is the most important test of the whole work: if it fails, the entire scientific result loses its force, because the difference between optimizers could be explained by uncontrolled randomness.

test_sweep_toy.py

```
def test_sweep_select_tunability():
    results = run_equal_budget_sweep(workload, budget, opt_spec, space, runtime,
                                     sweep_seed=0, store=None)

    assert len(results) == 3
    assert all("lr" in r.hparams for r in results)
    best = select_best_by_val(results)
    assert best.best_val == min(r.best_val for r in results)
    rows = tunability_table(results)
    assert [r["lr"] for r in rows] == sorted(r["lr"] for r in rows)
```

An integration test of the whole tuning pipeline on a toy task: sweep → selection → table. `store=None` disables caching — the test must not touch the filesystem and must not depend on what is left over from previous runs.

test_aggregate.py

```
def test_bootstrap_ci_constant():
    assert bootstrap_ci([2.0, 2.0, 2.0]) == (2.0, 2.0)

def test_bootstrap_ci_brackets_mean():
    lo, hi = bootstrap_ci([1.0, 2.0, 3.0, 4.0, 5.0], seed=0)
    assert lo <= 3.0 <= hi

def test_steps_to_target_first_crossing():
    assert steps_to_target_from_history(_HIST, 3.20) == 300
    assert steps_to_target_from_history(_HIST, 3.30) == 200
    assert steps_to_target_from_history(_HIST, 3.50) == 100
```

Note the comment in the test code: "first eval step where val_loss <= target, not the best/last." The test pins down the semantics of the metric, not just its computation. If someone mistakenly reworked the function into "the step with the best value," the test would catch it.

test_landscape.py

The most substantive test file: it checks mathematical properties.

```
def test_filter_normalize_matches_param_norm_per_tensor():
    d = filter_normalize(random_direction(params, g), params)
    for di, pi in zip(d, params):
        if pi.dim() >= 2:
            assert abs(di.norm().item() - pi.norm().item()) < 1e-4
        else:
            assert di.norm().item() == 0.0
```

A direct check of the definition: for matrices the norm of the direction equals the norm of the parameter, and for one-dimensional parameters the direction is zero.

```
def test_surface_center_equals_solution_and_restore():
    coords, Z = loss_surface_2d(model, eval_loss, theta, dx, dy, [-0.1, 0.0, 0.1])
    set_params_(model, theta); ref = eval_loss()
    assert abs(Z[1, 1] - ref) < 1e-5          # grid centre = the solution itself
    assert all(torch.equal(p.detach(), t)
                for p, t in zip(model.parameters(), theta))  # weights restored
```

Two invariants: (a) the point (0,0) of the grid gives exactly the same loss as the solution itself — meaning the offset is applied correctly; (b) after the function returns, the model's weights are restored. The second is protection against the most insidious class of errors, where a visualization function silently spoils the model and the next plot is built from the spoiled solution.

Analogously, `test_interpolation_endpoints_and_restore` checks that at $t=0$ you get solution A and at $t=1$ solution B.

Results: how to read them

What is in results/

```
results/  
├─ tables/    12 CSVs: 3 for each of the 4 experiments  
├─ figures/   11 PNGs: curves, tunability, three RQ3 figures  
└─ runs/     the JSON cache (absent in this copy)
```

The absence of `results/runs/` means the tables cannot be recomputed without retraining. For submitting the artifact this is fine (the cache can be large), but it is worth knowing.

RQ1: the main table (GPT-2 + LoRA, 5 seeds)

Optimizer	best LR	val_loss	$\pm$ std	steps to 3.20	ms/step	state, MB
AdamW	2.96e-3	3.1763	0.0021	400 (5/5)	241	2.36
SGD+momentum	4.44e-1	3.1813	0.0032	520 (5/5)	176	1.18
Sophia	5.82e-4	3.1819	0.0013	500 (5/5)	383	2.36
Lion	6.66e-4	3.1881	0.0019	540 (5/5)	100	1.18

How to interpret this. The spread between the best and the worst is 0.012 in loss units, i.e. about 1 % of perplexity. That is very little. The work's main conclusion, consistent with Choi et al. and AlgoPerf: **under fair tuning and an equal budget, the difference in quality between optimizers practically disappears.**

SGD+momentum is especially telling: the simplest method of the 1980s, at LR = 0.44, comes second and is statistically indistinguishable from AdamW ($p = 0.056$). This directly refutes the common claim that "SGD does not work for transformers" — it is true only when the learning rate is chosen incorrectly.

RQ1: significance

Pair	U	p
adamw vs sgd_momentum	3.0	0.0556
adamw vs lion	0.0	0.0079
adamw vs sophia	0.0	0.0079

$U = 0$ means complete separation of the samples: each of AdamW's five seeds is better than each of the competitor's five seeds. $p = 0.0079$ is the minimum attainable value for 5 versus 5. The correct formulation: "AdamW significantly outperforms Lion and Sophia statistically, but the effect size (0.006–0.012 in loss) is practically negligible." Statistical significance and practical importance are different things, and in this case they diverge.

RQ2: tunability

This is where the work's main practical result lies. From the table

`rq1_gpt2_lora_rq2_tunability.csv` and the right-hand panel of the plot:

- **AdamW** has the widest "basin": over almost a whole decade of LR (from $1.3\text{e-}3$ to $6.7\text{e-}3$) the quality stays within 0.01 of the optimum. This is exactly the reason for its popularity: it forgives a mistake in the choice of LR.
- **Lion** has a sharper optimum; a deviation of an order of magnitude up or down noticeably worsens the result.
- **Sophia** has an asymmetric basin: gentle on the left, a cliff on the right. At LR = $6.7\text{e-}3$ the loss is 6.26, at $1\text{e-}2$ it is 11.55 — i.e. complete divergence. This makes Sophia a risky choice without careful tuning.
- **SGD+momentum** works, but in a range that differs by 2–3 orders of magnitude. If you apply the "habitual" LR of about $1\text{e-}3$ to it, you get 3.607 — noticeably worse than everyone. This is where the myth of SGD's unsuitability comes from.

The practical conclusion is formulated like this: **an optimizer should be chosen not by the attainable minimum, but by the width of the region in which it attains that minimum.**

Cost: step time and memory

The ratio of step times (100 : 176 : 241 : 383 ms for Lion : SGD : AdamW : Sophia) must be interpreted with caution. AdamW and SGD in PyTorch are implemented as fused (fused/foreach) operations in C++/CUDA, Lion is taken from a third-party library, and Sophia in this project is written as a pure Python loop over parameters plus an extra `update_hessian` call. That means the figure of 383 ms reflects, first and foremost, the quality of the implementation rather than the algorithmic complexity of the method. This limitation is worth stating explicitly in the text of the work.

Peak memory is almost identical (3512 versus 3513 MB), because with LoRA only 0.3 % of the parameters are trainable, and the optimizer state is lost against the background of activations. The meaningful column here is `opt_state_mb`, and it reproduces the theory exactly (1× versus 2×).

The transferability check on SmolLM2-360M

Optimizer	val_loss	rank on SmolLM2	rank on GPT-2
AdamW	2.4370	1	1
Sophia	2.4421	2	3
Lion	2.4461	3	4
SGD+momentum	2.4767	4	2

AdamW keeps first place, Sophia and Lion shift by one position, and SGD drops from second place to last. That is, **the ranking transfers only partially**. This is exactly the outcome the config declared admissible in advance, and it supports the AlgoPerf thesis: conclusions obtained on one task cannot be carried over to others without checking.

Worth noting separately: on SmolLM2 SGD's best LR is 0.215, and the next grid point (0.464) gives a loss of 6.10 — i.e. divergence. SGD's optimum turned out to be almost at the edge of a cliff, which underlines once again its sensitivity to LR on this architecture.

RQ3: the landscape

Naive versus filter normalization. On one and the same AdamW solution, with the same total step size, naive normalization gives a loss range up to 5.4, while filter normalization gives up to 1.9. The minimum "looks" almost three times sharper purely because of how the directions were chosen. This is the demonstration of the thesis: **a landscape picture without correct normalization is not evidence**.

Per-optimizer slices. The filter-normalized slices of the three solutions are comparable with one another, and from them the difference in the shape of the minimum's neighbourhood is visible.

Interpolation. Between the AdamW and SGD+momentum solutions a barrier is found (about 0.027, per the figure caption). This means that the two optimizers, having started from one and the same point on the same data, arrived in **different basins** of the loss function, not at different points of the same one. This is a substantive fact: similar final quality does not mean similar solutions.

Three end-to-end scenarios

Scenario 1:

```
python scripts/run_rq1.py --config configs/experiments/rq1_gpt2_lora.yaml
```

1. `load_experiment` reads the YAML → `ExperimentConfig` (all dataclasses frozen).
2. `ArtifactStore("results/runs")` creates the cache root.
3. `capture_env()` prints the versions and the GPU model.
4. `run_rq1` begins the loop over the four optimizers. For each: - `sample_trials` builds 18 LR values on a logarithmic grid; - for each trial: if `results/runs/rq1_gpt2_lora/<opt>/sweep/trialNN.json` exists — load it; otherwise — `train_one`; - inside `train_one`: the seed is fixed → GPT-2 is loaded → LoRA is attached → WikiText-2 is tokenized → a `Trainer` is created → the optimizer is built from the name and LR → the schedule is built → `fit` runs 800 steps, evaluating every 100 → a `RunResult` is returned → it is saved to JSON; - `select_best_by_val` takes the trial with the minimum `best_val`; - `run_multiseed` re-runs that configuration on seeds 0-4 (also with caching).

5. The upshot is the `out` dictionary with $18 + 1 + 5$ results per optimizer.
6. `write_tables` computes the statistics, recomputes time-to-target from the histories, and writes three CSVs.
7. `plot_tunability` and `plot_val_curves` draw two PNGs.
8. A summary is printed.

Total training: $4 \times (18 + 5) = 92$ runs of 800 steps. At ~ 240 ms per step that is on the order of 5 hours of GPU time — hence the importance of caching.

Scenario 2: `python scripts/run_landscape.py`

1. Three times: seed 0 is fixed → a toy transformer and character data are built → 400 steps are trained with the given optimizer → the weights are copied into `solutions[name]`.
2. The first model becomes the "host"; the `eval_loss` closure is created.
3. Figure A: for each solution, two filter-normalized directions, a 25×25 grid (625 validation evaluations per panel).
4. Figure B: one solution (AdamW), one pair of raw directions, two normalizations, two grids.
5. Figure C: 31 interpolation points between AdamW and SGD.
6. In total roughly 2500 full validation passes — on a toy model that is minutes on CPU.

Scenario 3: `python -m pytest -q`

Eight files, all on CPU, all in seconds. The order of checks by meaning: the configuration loads → the LR grid is correct → the optimizers reduce the loss → the trainer works on both tasks → two identical runs match bit for bit → the sweep and the selection work → the statistics are computed correctly → the landscape primitives are mathematically correct and restore the model.

Strengths and weaknesses: what to say at the defence

What was done well

1. **The fairness invariant is encoded in the types**, not left to discipline. One workload, one budget for the whole experiment.
2. **The same seed across the entire sweep** — the difference between trials is explained by LR alone.
3. **Separation of "selection" and "evaluation"**: the best LR is chosen on the sweep seed, and quality is measured on other seeds.
4. **Resumability and idempotence**: any run can be interrupted and continued, and the analytics can be recomputed without training.

5. **Recomputing time-to-target from history** — the threshold can be recalibrated honestly, and the rule for choosing it is stated explicitly.
6. **The departure from the Sophia paper is documented right in the code**, not hidden.
7. **The methodology's weak spots are stated in the comments** — including the unsuccessful first choice of threshold (3.36).
8. **The tests check invariants rather than implementation details**: restoration of the model after slicing, bit-exact reproducibility, the semantics of the first threshold crossing.

What should be acknowledged as a limitation

1. **Only the learning rate is tuned**. Moments, weight decay and rho are fixed at defaults. This is uniform, but Choi et al. show precisely that fixing secondary hyperparameters systematically biases the comparison — usually in favour of the simpler methods.
2. **Different default weight-decay values across optimizers** (Sophia 0.1 versus AdamW 0.01) — an unintentional asymmetry inherited from the method authors' recommendations.
3. **Step time is not an algorithmic comparison** — it reflects implementation quality (PyTorch's fused kernels versus a Python loop).
4. **With K = 3 the Mann-Whitney test cannot give $p < 0.05$ for any outcome**; the significance table for SmolLM2 shows the floor of the statistic, not "near significance."
5. **The granularity of time-to-target is 100 steps** (the evaluation period). A difference of "400 versus 520" is one or two evaluation points.
6. **The cache is addressed by trial index rather than by hyperparameter value**: changing the grid without clearing `results/runs/` gives silently incorrect results.
7. **The Sophia implementation has not been verified against a reference one**; the test only checks that "the loss goes down."
8. **RQ3 was carried out on a toy model** — the conclusion about misleading visualization is methodological and transferable, but the specific shapes of the neighbourhoods do not apply to LLMs.
9. **Truncating the corpus (2500 blocks)** means the model passes over the data about 2.5 times; the conclusions apply to a short fine-tuning regime, not to long training.

What could be improved (good answers to "so what's next?")

- Key the cache by a hash of the hyperparameters instead of an index.
- Add config validation: check that every optimizer has a search space, that the ranges are non-empty, and that the threshold is attainable.
- Extend the search to a second hyperparameter (weight decay, for instance) for at least two optimizers, to gauge how strongly fixing it biases the result.
- Verify SophiaG against a reference implementation on a small problem.

- Increase the evaluation frequency in the vicinity of the threshold, so that time-to-target is measured more precisely.
 - Replace the single `eval_every_steps` value with a logarithmic grid of evaluations.
-

Glossary

Autocast — a mode in which individual operations are performed in reduced precision (bf16), while the weights and gradients stay in fp32.

bf16 (bfloat16) — a 16-bit format with the same exponent range as fp32 but a smaller mantissa. Unlike fp16, it does not require gradient scaling.

Bootstrap — a method of estimating uncertainty by repeated resampling with replacement from the data at hand.

Filter normalization — a way of normalizing random directions when visualizing the landscape: each parameter tensor gets a direction of its own norm.

Gradient accumulation — accumulating gradients over several small batches before a single optimizer step; a way to obtain a large effective batch under limited memory.

LoRA (Low-Rank Adaptation) — fine-tuning via a small low-rank correction to frozen weights; a fraction of a percent of the parameters is trained.

Packing — gluing the corpus into one token sequence and cutting it into fixed-length blocks, without padding.

Perplexity — $\exp(\text{mean loss})$; interpreted as "the effective number of equally likely options for the next token."

Steps-to-target — the number of steps until a given metric threshold is first reached; an AlgoPerf-style speed metric.

Tunability — how wide the range of a hyperparameter is within which the method gives a result close to optimal.

Warmup — the initial stretch of training with a linearly increasing learning rate.

Self-check exercises

1. Find every place in the code that guarantees that two optimizers see batches in the same order. (Hint: there are three of them, in different files.)
2. What will happen if you swap `set_global_seed(seed)` and `build_experiment_unit(...)` in `run_one.train_one`? Which result of the work exactly becomes invalid?

3. In `sweep_runner` the cache is addressed as `trial00.json`, `trial01.json`. Write a variant that is addressed by the learning-rate value, and explain which error this prevents.
4. Why is `update_hessian()` called before `clip_grad_norm` and not after? What would change in the curvature estimate with the reverse order?
5. In `evaluate` the loss is weighted by the number of tokens. Construct an example where a plain average over batches would give a different answer. Does such a situation arise in this project?
6. Why does `steps_to_target` return `None` rather than `max_steps` when the threshold is not reached? How would the `steps_to_target_mean` column change if `max_steps` were substituted for `None`?
7. Explain why with 3 seeds the Mann-Whitney test cannot give $p < 0.05$. What is the minimum number of seeds needed?
8. In `filter_normalize`, one-dimensional parameters get a zero direction. What would be visible in the landscape picture if they were not zeroed?
9. Build a table: which `RunResult` fields are used in `write_tables`, which in the plots, and which are not used anywhere.
10. Add a fifth optimizer to the project (RMSprop, for example). List all the files that would have to be changed. (The correct answer: two.)